

AWS DATA ENGINEER

S3 | Glue | Redshift | Athena | IAM | Kinesis | Lambda | CloudWatch

Prepared by Sindhura Basavaraju - [LinkedIn](#)

Current as of September 2026.

Contents

Contents	2
0 The One-Page Map	4
1 Amazon S3	5
1.1 What it does and where it fits	5
1.2 Key components a senior engineer should know	6
1.3 Scenario questions	10
2 AWS Glue	12
2.1 What it does and where it fits	12
2.2 Key components	13
2.3 Scenario questions	15
3 Amazon Redshift	17
3.1 What it does and where it fits	17
3.2 Key components	17
3.3 Scenario questions	20
4 Amazon Athena	23
4.1 What it does and where it fits	23
4.2 Key components	23
4.3 Scenario questions	27
5 AWS IAM	29
5.1 What it does and where it fits	29
5.2 Key components	29
5.3 Scenario questions	32
6 Amazon Kinesis	35
6.1 What it does and where it fits	35
6.2 Key components	36
6.3 Scenario questions	40
7 AWS Lambda	43
7.1 What it does and where it fits	43
7.2 Key components	43
7.3 Scenario questions	46
8 Amazon CloudWatch	48

8.1 What it does and where it fits	48
8.2 Key components	48
8.3 Scenario questions	52
9 End-to-End Data Architecture	54
9.1 The reference architecture	54
9.2 The core decisions	54
9.3 Scenario questions (whiteboard style)	58
A Appendix: Night-Before Pack	61
A.1 Numbers worth knowing cold	61
A.2 Ten traps interviewers set	62
A.3 Twenty-minute verbal drill	62
A.4 Sources checked for this guide	62

The One-Page Map

Every question in a senior data engineering interview is really one of four questions:

1. Where does data land?
2. How does it get transformed?
3. How do people query it?
4. How do you keep it correct, secure and cheap?



IAM wraps all of it. Every arrow above is an IAM role with a policy attached.

Key AWS Data Engineering Tools Quick Reference Guide (2026)

Key concepts. Real use cases. What to focus on.

Tool	What it does / One-line description	Why it's useful (in data pipelines)	Key things to know in 2026
 S3	Scalable, durable object storage used as the data lake.	Stores all raw and processed data reliably at low cost and separates storage from compute.	S3 Tables (managed Apache Iceberg), Express One Zone, Intelligent-Tiering, lifecycle policies, access control with IAM/Lake Formation.
 AWS Glue	Serverless data integration service for ETL, data catalog and metadata management.	Simplifies data preparation and ETL at scale without managing infrastructure.	Glue 5.0/6.0 (Spark 3.5+), Python 3.12+, Ray jobs, job bookmarks, data quality, Apache Iceberg support, and lower DPU pricing.
 Redshift	Managed, high-performance data warehouse for analytics and business intelligence.	Runs fast analytical queries on large datasets and integrates with the data lake and BI tools.	RA3 instances, Serverless, query acceleration, zero-ETL from Aurora/RDS, materialized views, Redshift Spectrum (query data in S3), Python UDFs deprecated (use SQL or Lambda UDFs).
 Athena	Serverless SQL engine to query data directly in S3 using standard SQL.	Great for ad-hoc analysis and data exploration without managing any infrastructure.	Supports Iceberg, Hudi and Delta; partition pruning; cost based on data scanned; ACID transactions with Iceberg; result reuse and managed query results.
 IAM	Manages access and permissions for AWS resources.	Ensures secure access using least privilege principle across your data platform.	Use IAM roles (not users), policy conditions, permission boundaries, cross-account access, Lake Formation for table/column level permissions, short-lived credentials (STS).
 Kinesis	Real-time data ingestion and stream processing service.	Captures and processes streaming data in real time for analytics, monitoring or downstream processing.	Kinesis Data Streams (real-time), Kinesis Data Firehose (deliver to S3/Redshift/OpenSearch), Kinesis Data Analytics (Apache Flink), on-demand mode, enhanced fan-out.
 Lambda	Serverless compute to run code in response to events.	Lets you run small, event-driven tasks without managing servers. Commonly used for automation, data transformation and event-based workflows.	Supports up to 10 GB memory and 6 vCPUs, 10–15 minute timeout, event filtering, Lambda SnapStart (for Java), and integration with many AWS services.
 CloudWatch	Monitoring, logging and alerting for AWS resources and applications.	Helps you monitor pipeline health, detect issues, set alerts and troubleshoot quickly.	CloudWatch Logs Insights (query logs with SQL), metrics and dashboards, alarms, EventBridge rules for automation, and improved observability for serverless and data services.

1 Amazon S3

The data lake is S3. Everything else is compute pointed at it.

1.1 What it does and where it fits

Amazon S3

Amazon S3 (Simple Storage Service) is an AWS object storage service used to store and retrieve data as objects inside buckets. S3 uses a flat object namespace, where each object is identified by a unique key. Path-like prefixes such as `raw/sales/2026/` provide a folder-like structure, but there are no traditional folders.(vs ADLS which has real file like system)

S3 is designed for extreme durability, massive scalability, and low-cost storage (Pay per GB). In a data engineering architecture, it acts as the central storage layer of the data lake as it can hold raw, cleaned, curated, staged, and archived data.

The idea of S3 is to decouple storage from compute: data can remain safely in S3 while different services such as Glue, Athena, EMR, and Redshift attach compute only when needed, without requiring the data to be moved into each service.

ANALOGY

S3 is a giant warehouse with numbered shelves. The shelf label (the key) can look like `sales/year=2026/month=09/file.parquet`, but the warehouse has no rooms; it is one enormous floor. Glue, Athena and Redshift are the forklifts and clerks you hire to work the floor; they never own the shelves.

S3 vs ADLS: S3 has a flat object namespace, so folder-like paths are just prefixes in the object key. ADLS Gen2 with Hierarchical Namespace provides real directory and filesystem semantics, which makes file and directory operations more natural for analytics workloads. The difference becomes noticeable when you start doing filesystem operations like move folder, rename etc and access control (IAM for S3 and POSIX for ADLS Gen 2)

- S3 = objects with paths. - path-like key/prefix → not a real folder structure.
- ADLS Gen2 HNS = files and folders - real hierarchical directories/files.
- **S3 Object store first.**
Paths are names that help organize objects.
- **ADLS Gen2 + HNS Object store + filesystem capabilities.**
Paths represent a real hierarchy of directories and files.

1.2 Key components

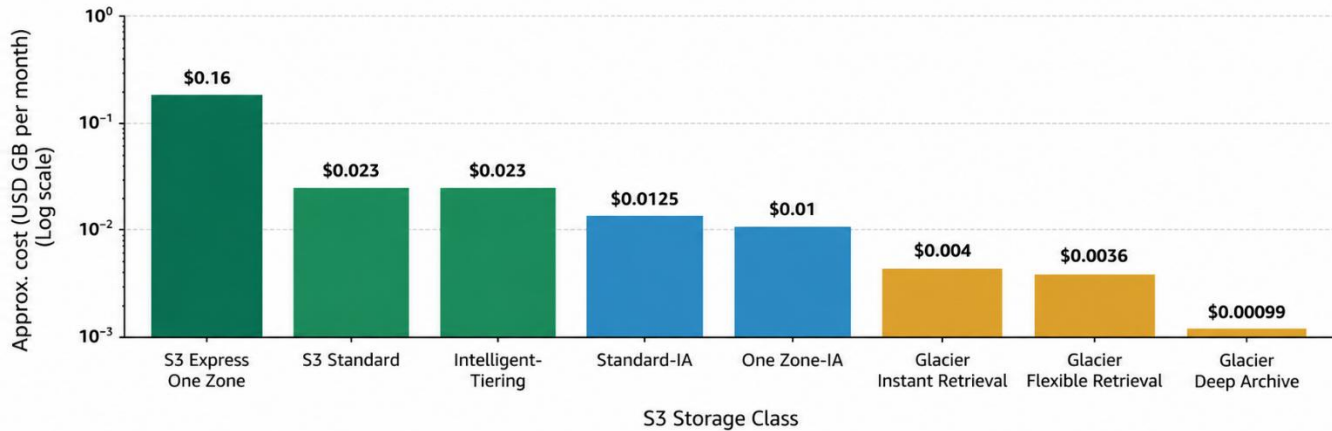
Buckets, objects, keys, consistency

- **Three bucket types:**
 - General purpose (the normal one, all storage classes except Express),
 - Directory buckets (Express One Zone only, single AZ, hundreds of thousands of requests per second per bucket)
 - Table buckets (S3 Tables: native Apache Iceberg tables with automatic compaction and snapshot expiry).
- **Strong read-after-write consistency** for all operations since December 2020. Strong consistency means that once S3 successfully creates, updates, or deletes an object, any subsequent read or list request immediately sees that latest change. If an interviewer mentions eventual consistency they are testing whether you are current.
- **Object size:** An S3 object can be uploaded in a single request up to 5 GB. For larger objects, S3 uses multipart upload, which splits the file into smaller parts and uploads them separately, supporting objects up to 5 TB. Multipart upload is commonly handled automatically by AWS SDKs and tools such as Spark. Unfinished multipart uploads can continue to incur storage charges, so it is good practice to use an S3 Lifecycle rule to automatically remove incomplete uploads.
- **Versioning:** S3 Versioning keeps previous versions of an object when the object is updated or deleted. This helps protect data from accidental changes or deletion and is also required for some replication scenarios. However, storing multiple versions increases storage costs, so you can use Lifecycle rules to automatically delete older, non-current versions.
- **Request rate:** S3 can handle at least 3,500 write requests (PUT, COPY, POST, DELETE) and 5,500 read requests (GET, HEAD) per second per prefix. S3 automatically scales to handle higher request rates and using multiple prefixes can increase overall throughput. You generally do not need random prefixes just to increase performance.

Storage classes and lifecycle

Storage class	Use it when...	Simple way to remember
S3 Standard	You access the data frequently. This is the normal/default choice.	Hot data = Standard
S3 Express One Zone	You need very fast access and very low latency for frequently used data, such as ML or interactive workloads, and the data can stay in one AZ.	Very fast + one AZ
S3 Intelligent-Tiering	You don't know how often the data will be accessed, or the access pattern changes over time. S3 automatically moves objects between access tiers.	Don't know the pattern → Intelligent-Tiering
S3 Standard-IA	Data is stored for a long time but accessed only occasionally, such as older data that is still needed sometimes.	Infrequent access + multiple AZs
S3 One Zone-IA	Same idea as Standard-IA, but you can accept storing the data in one AZ to reduce cost.	Infrequent + one AZ
S3 Glacier Instant Retrieval	You need archive storage, but still need to retrieve the data within milliseconds when required.	Archive + instant access
S3 Glacier Flexible Retrieval	Data is rarely accessed, such as backups and archives, and you can wait minutes to hours for retrieval.	Archive + okay to wait
S3 Glacier Deep Archive	Data must be kept for years and is almost never accessed, such as long-term compliance records.	Keep for years + almost never read

S3 Storage Classes: Cost drops as access speed and minimum duration constraints grow



Key takeaway: Storage cost decreases as access becomes less frequent, and minimum storage duration and retrieval time increase. Choose the right storage class based on your data's access pattern, retrieval needs, and retention requirements.

Note: Prices are approximate (us-east-1) and may vary by region.

Figure: Storage class spectrum. Green = hot, blue = infrequent, amber = archive. Prices are approximate us-east-1 list prices.

Lifecycle rules are automated rules that manage objects over time. They can move objects to cheaper storage classes after a certain number of days and delete objects when they are no longer needed. You can apply lifecycle rules based on things like object age, prefix, tags, or object size. They can also clean up old object versions and incomplete multipart uploads.

Interviewers care more about the reasoning behind the storage-class choice than simply remembering the class names. The key trade-off is always storage cost vs retrieval cost vs minimum storage duration vs availability. The right choice depends on the data's access pattern-how often it is read, how quickly it needs to be retrieved, how long it will be stored, and whether single-AZ storage is acceptable. When the access pattern is unknown or changes frequently, **S3 Intelligent-Tiering** is a good choice because AWS can automatically move objects between access tiers based on how they are used.

Data lake architecture on S3

Zone (medallion name)	Contents	Format and rules
Raw / Landing (Bronze)	Data exactly as it arrives from the source. It is kept as the original copy so you can always go back and reprocess it.	Keep the original source format such as JSON, CSV, or Avro. Organise by ingestion date. Data should be append-only and normally not overwritten.
Cleaned / Conformed (Silver)	Cleaned and validated data with a consistent schema. Duplicates are removed, data types are standardised, and business rules are applied.	Usually, Parquet or Iceberg. Partition based on useful business/query columns. This is the trusted dataset used for downstream processing.

Zone (medallion name)	Contents	Format and rules
Curated / Serving (Gold)	Business-ready data such as aggregations, reporting tables, dimensional models, and feature tables.	Usually Parquet or Iceberg, optimised for consumers such as Athena, Redshift, BI tools, or ML workloads. Aim for fewer, reasonably large files.
Sandbox / Scratch	Temporary space for analyst exploration, testing, and ad-hoc work.	Flexible format. Data should have a short lifecycle and be automatically deleted when no longer needed.

Open table formats

Open table formats such as Iceberg, Delta Lake, and Hudi sit on top of files like Parquet and add database-like features such as ACID transactions, schema evolution, and time travel. They also manage how data is organised and queried, so the query engine can prune unnecessary data without the user needing to know the physical file layout.

On AWS, Apache Iceberg is increasingly the preferred open table format because it integrates well with services such as S3, Glue, Athena, Redshift, and EMR. Different AWS services may support different Iceberg specification versions, so when using newer Iceberg features, it is important to check which version each service supports and whether those features work consistently across your entire data platform.

Partitioning on S3

Partitioning organises data by commonly filtered columns, usually date, so Athena, Glue, and Spark can scan only the relevant data.

- Choose partitions based on query patterns such as date, region, tenant, etc.
- Avoid high number of partitions, which can slow metadata and query planning.
- Avoid tiny files; aim for roughly 128 MB–1 GB Parquet files.
- Use compaction to combine small files.
- Partition projection (Athena) and hidden partitioning (Iceberg) reduce the need to manually manage partitions.

Security: IAM, bucket policies, encryption

Control	What it is	When to use
IAM identity policy	A policy attached to a user or role that defines what that identity can do, such as s3:GetObject on specific S3 resources.	Controlling access for Glue jobs, Lambda, EC2, users, etc.
Bucket policy (resource-based)	A policy attached directly to an S3 bucket that defines which principals can access the bucket and what they can do.	Cross-account access and bucket-wide security rules.

Control	What it is	When to use
Access Points	Named S3 endpoints, each with its own access policy, that provide different ways for different applications or teams to access the same bucket.	Useful when many teams or applications share one large bucket.
Block Public Access	S3 settings that prevent public access even if another policy accidentally allows it.	Keep enabled for data lakes unless there is a specific, controlled reason not to.
Lake Formation	Provides fine-grained permissions for data in the Glue Data Catalog, including table and column-level access.	When you need analytics users to access specific tables or columns rather than managing everything with S3 policies.

S3 Encryption:

Encryption at rest protects your data while it is stored in S3. By default, S3 uses SSE-S3, where AWS manages the encryption keys. For more control, you can use **SSE-KMS**, where keys are managed through AWS KMS and you get more control over access and auditing. S3 Bucket Keys can reduce the number of KMS requests and therefore reduce KMS costs.

DSSE-KMS provides an additional layer of encryption for workloads with stricter security or compliance requirements. **Client-side encryption** means the data is encrypted before it is uploaded to S3, so the encryption happens on the client side rather than in S3.

“I’d encrypt with KMS” without mentioning that every reader role also needs the required kms: Decrypt permission on that KMS key. A Glue job or Redshift COPY can have the correct S3 permissions but still fail with AccessDenied because it cannot use the KMS key. This is especially important in cross-account setups, where both S3 permissions and KMS key permissions must allow access.

Incremental processing patterns on S3

Pattern	How it works	Best for / When to use
 Partition-based	Process only new or changed partitions, typically based on a date column (e.g., year/month/day). Track the last processed partition (e.g., in a metadata table) and process only newer partitions.	Batch workloads, daily/periodic processing. Simple to implement and cost effective for large datasets.
 Event-driven	Use S3 Event Notifications to trigger services like Lambda, EventBridge or SQS when new objects are created (e.g., ObjectCreated event) and process the files.	Near real-time or per-file processing. Good for event-driven architectures and low-latency use cases.
 Glue Job Bookmarks	AWS Glue tracks which files have already been processed using job bookmarks (based on file path or modification time) and processes only new or updated files in subsequent runs.	Glue-based ETL jobs reading from S3 or JDBC. Good for incremental loads without managing custom metadata.
 S3 Inventory + manifest	S3 Inventory generates a daily or weekly list of all objects in a bucket. Compare with a previous inventory or use a manifest file to identify new or changed files.	Very large buckets where LIST operations are slow or expensive. Useful for audits, data reconciliation and batch processing.
 Iceberg incremental reads	Query only the changes between snapshots using Iceberg's metadata (e.g., snapshot IDs, time travel or incremental reads) instead of scanning the entire dataset.	Modern data lakehouse architectures using Apache Iceberg (e.g., with Athena, Glue, EMR or Redshift). Efficient incremental processing and CDC-style workloads.

1.3 Scenario based questions

Q1. Your S3 data lake has millions of small Parquet files, and your Spark jobs are becoming slow. What would you do?

I would first identify why the files are small and then introduce a compaction process to combine them into larger files, typically around 128 MB–1 GB. I would also check the partitioning strategy to make sure we are not creating too many partitions. For example, partitioning by day may be better than partitioning by hour if the hourly partitions contain very little data.

Remember: Small files → too many file operations → slower processing. Compact them.

Q2. A Glue job processes the same S3 files every day instead of only processing new files. How would you fix it?

I would use an incremental processing strategy instead of scanning the entire dataset. If the data is partitioned by date, the job can process only the new partitions. For Glue jobs, I could also use Glue Job Bookmarks to track which files have already been processed. For more event-driven workloads, S3 events could trigger processing when new files arrive.

Remember: Don't reread everything → identify and process only new data.

Q3. Your company accidentally deleted important files from an S3 bucket. How would you protect against this?

I would enable S3 Versioning so deleted or overwritten objects can be recovered. I would also use appropriate IAM permissions and S3 Block Public Access to reduce the chance of accidental or unauthorized changes. For critical data, I would consider cross-region replication or backups depending on the recovery requirements. Lifecycle rules can be used to eventually remove old versions to control storage costs.

Remember: Versioning protects against accidental deletion and overwriting.

Q4. Athena is scanning terabytes of data for a simple query, and the query cost is very high. How would you optimize it?

I would first check whether the data is stored in a columnar format such as Parquet instead of JSON or CSV. Then I would partition the data based on common query filters, such as event date or region. I would make sure queries use partition filters and select only the required columns instead of `SELECT *`. I would also compact small files because too many small files can hurt performance.

Remember: Parquet + partitioning + column pruning + good file sizes = less data scanned.

Q5. A Glue job can access an S3 bucket, but it fails with Access Denied when reading encrypted files. What would you check?

I would first check the IAM role's S3 permissions, such as `s3:GetObject`. If the objects use SSE-KMS, I would then check whether the Glue role has permission to use the KMS key, especially `kms:Decrypt`. I would also check the KMS key policy because the key policy must allow the role to use the key. In a cross-account setup, both the S3 permissions and KMS permissions need to be correctly configured.

Remember: For SSE-KMS, you need both S3 access + KMS access.

MENTAL MAP | S3 = DURABLE + CHEAP + DECOUPLED

S3

STORE

- Objects are stored as keys (not traditional folders)
- 11 nines (99.999999999%) durability
- Strong read-after-write consistency (for PUT, GET and LIST)
- Up to 5 TB per object (use multipart upload for large files)

STORAGE CLASSES

- **Hot** (frequent access): S3 Standard, S3 Express One Zone
- **Unknown access pattern**: S3 Intelligent-Tiering
- **Infrequent access**: S3 Standard-IA, S3 One Zone-IA
- **Archive**: S3 Glacier Instant Retrieval, S3 Glacier Flexible Retrieval, S3 Glacier Deep Archive

CLASS TRADE-OFF

- Consider storage cost per GB, retrieval cost, minimum storage duration and availability / AZ requirements.

LIFECYCLE

- Transition objects to cheaper storage classes, expire old data, delete non-current versions, and clean up incomplete multipart uploads.

DATA LAKE ZONES

- **Raw / Bronze**: original, immutable data
- **Cleaned / Silver**: validated, transformed data (e.g., Parquet or Iceberg)
- **Curated / Gold**: business-ready data for analytics and downstream consumption
- Typically use 1 bucket per zone per environment (dev / test / prod).

DATA LAYOUT

- Partition by commonly filtered columns (e.g., event date, region)
- Use appropriate file sizes ($\approx$ 128 MB – 1 GB)
- Avoid small files (too many files hurt performance and cost)
- With Iceberg, partitions are managed automatically and hidden from users.

SECURITY

- **IAM identity policy**: permissions for your own users/roles
- **S3 bucket policy**: control access for other accounts, cross-account access and explicit deny rules (e.g., enforce TLS)
- **AWS Lake Formation**: fine-grained table/column/row-level access for analytics

ENCRYPTION

- **SSE-S3**: default encryption (AES-256, keys managed by S3)
- **SSE-KMS**: customer managed key, gives audit trails and fine-grained access control (readers need `kms:Decrypt`)
- **S3 Bucket Keys**: reduce KMS request costs

INCREMENTAL PROCESSING

- Partition-based processing (process only new partitions)
- S3 events $\rightarrow$ EventBridge / SQS / Lambda for event-driven workloads
- AWS Glue job bookmarks to track processed files
- S3 Inventory to identify new or changed files at scale
- Iceberg snapshots for incremental reads (e.g., time travel, CDC-style)
- Make writes idempotent (e.g., partition overwrite, MERGE or deterministic keys)

2 AWS Glue

Serverless Spark plus the account-wide metadata catalog

2.1 What it does and where it fits

AWS Glue is essentially three main things under one service name.

- The Glue Data Catalog is a central metadata store that keeps information about databases, tables, schemas, partitions, and where the underlying data is stored.
- Glue Crawlers scan sources such as S3 or JDBC databases and automatically discover schemas and update the Glue Data Catalog.
- Glue ETL Jobs are managed data-processing jobs, typically using Apache Spark, Python Shell, or Ray, so you can run transformations without managing your own servers or Spark clusters.

Around these are Workflows, Triggers, Job Bookmarks, and Glue Studio, which help with scheduling, orchestration, incremental processing, and visual job development.

In a typical data pipeline, the Data Catalog provides the metadata layer between the data and analytics services such as Athena, Redshift Spectrum, EMR, and Lake Formation. Glue ETL jobs are commonly used to transform data from Bronze → Silver → Gold, especially when you want a managed Spark environment without operating EMR or Databricks.

ANALOGY

Think of the Glue Data Catalog as a library catalogue. It does not store the actual books (the data lives in S3); it simply keeps track of where the books are, what they contain, and how they are organised so that anyone can find and use them.

Glue Crawlers are like librarians who walk through the library, examine the books, and update the catalogue whenever new books appear or existing one's change.

Glue ETL Jobs are like temporary workers who come in, reorganise the books, clean them up, combine collections, create summaries, and place them on different shelves. Once the work is finished, they leave. You only pay for them while they are working, and you never have to manage the underlying infrastructure yourself.

2.2 Key components

Crawlers and the Data Catalog

- A Glue Crawler is a scanner that builds the catalog for you. You point it at a data source (an S3 path or a JDBC database), give it an IAM role so it can read that source, and optionally a schedule so it re-runs automatically. It looks at the files, works out the schema (using built-in or custom classifiers to recognise formats like CSV, JSON and Parquet), and then creates or updates the matching tables in a Glue Data Catalog database.
- For S3 sources it also understands Hive-style folder names such as year=2026/month=09, and registers each of those folders as a partition of the table. To keep re-crawls cheap on large buckets, you can set it to crawl only new folders instead of rescanning everything.
- The Glue Data Catalog stores metadata, not the actual data. It contains information such as columns, data types, partition keys, table properties, file formats and S3 locations. Services such as Athena, Redshift Spectrum, EMR, and Lake Formation can use this metadata to discover and access the underlying data.
- The Data Catalog is regional and account specific. Cross-account access can be enabled using resource policies or Lake Formation, depending on the architecture.
- Glue Schema Registry is separate from the Data Catalog. It is mainly used to manage and validate schemas for streaming data such as Kinesis and Amazon MSK, including schema evolution for formats such as Avro, JSON Schema, and Protobuf.

DynamicFrames

A DataFrame has one fixed schema, so a column that is a string in one file and an int in another fails the read. A DynamicFrame is Glue's wrapper that lets each record carry its own schema, turning the mismatch into a "choice type" you settle with `resolve Choice (cast, make_struct, make_cols or project)`, then `toDF()` for normal Spark. DynamicFrames also give you catalog-aware reads and writes, job bookmarks and relationalize for flattening nested JSON, but they miss out on Catalyst optimisation. So the working pattern is DynamicFrames at the edges (read from catalog, fix schema drift, write with bookmarks) and DataFrames in the middle for joins, windows and aggregations.

	DynamicFrame	DataFrame
Schema	Inferred per record, supports choice types	Fixed, inferred once or supplied
Schema drift	Tolerates, resolve later	Fails or nulls
Optimiser	Limited; converts to DF for heavy work	Catalyst + AQE, full Spark SQL
Bookmarks	Native support via <code>transformation_ctx</code>	Not directly
Best at	Ingest from catalog, semi-structured, writes back to catalog	Transformations, joins, aggregates

Job bookmarks and incremental processing

A job bookmark is Glue's memory of what a job has already processed, so each run only picks up new data instead of reloading everything. For S3 it remembers which files it has seen (by path and last-modified time). To make it work you turn bookmarks on in the job settings and give every `DynamicFrame` you want tracked a fixed `transformation_ctx` name. The bookmark only saves at `job.commit()` at the end of the run, so if the job fails partway the next run re-reads the same data; that is why your writes need to be idempotent (safe to repeat). You can pause a bookmark to reprocess data without moving it forward, or reset it to start from scratch. Two limits to remember: on S3 a changed file is only picked up if its modified time changes and the whole file is then re-read

Workflows, triggers, orchestration

- Triggers start Glue jobs or crawlers. They can run on a schedule, manually/on demand, when an EventBridge event occurs, or based on the success/failure of another job or crawler.
- Workflows connect multiple Glue jobs, crawlers, and triggers into a simple DAG. They are useful when the pipeline is mostly within AWS Glue and you want a managed, low-operations solution.
- For more complex orchestration, use Step Functions or MWAA (Managed Airflow). Step Functions is serverless and works well for cross-service workflows, branching, and retries; Airflow provides the most flexibility for complex pipelines but requires more operational management.

Performance and cost optimisation

Lever	What to do	Why
Worker type and count	Choose the right worker type and size based on data volume and workload. Use Auto Scaling when appropriate.	Balances performance and cost. Avoids paying for unnecessary workers.
Glue version	Move to 6.0. Use a current, supported Glue version with a compatible Spark/Python runtime.	Newer versions generally provide better performance, features, and support.
Flex execution	Use Flex for non-urgent batch jobs	Can reduce cost, but jobs may wait for spare capacity before starting.
Input layout	Prefer Parquet/ORC , partition by commonly filtered columns, and use partition pruning.	Reads less data, reducing runtime and cost.
Small files	Avoid many tiny files. Use <code>groupFiles / groupSize</code> when reading S3 and compact output files when needed.	Reduces file-listing and task-scheduling overhead.
Shuffle	Reduce unnecessary shuffles, broadcast small lookup tables, and handle skewed keys when necessary.	Reduces data movement and prevents slow tasks caused by skew.
Bookmarks and pruning	Use Glue Job Bookmarks or other watermark/partition logic to process only new or changed data.	Avoids full reloads and reduces runtime and cost.
Observability	Use Glue metrics, CloudWatch logs/alarms, and Spark UI for troubleshooting.	Helps identify slow stages, failures, high resource usage, and bottlenecks.
Timeouts and retries	Set appropriate job timeouts and retry limits. Make writes idempotent.	Prevents runaway jobs and allows safe recovery from temporary failures.

Lever	What to do	Why
Cost optimisation	Right-size workers, use Auto Scaling/Flex where appropriate, optimise file formats and partitions, and avoid unnecessary jobs/crawlers.	Reduces Glue processing costs while maintaining performance.

2.3 Scenario based questions

Q1. Your Glue job processes 5 TB of S3 data every night, but only 2% is new. How would you optimize it?

I would avoid a full scan and implement incremental processing. If the data is partitioned by date, I would process only the new partitions. I could also use Glue Job Bookmarks to track previously processed files. For event-driven ingestion, S3 events/EventBridge could trigger the Glue job when new data arrives.

Remember: Don't process 5 TB when only 2% changed.

Q2. Your Glue job is taking much longer than expected. How would you troubleshoot it?

I would check the Spark execution plan and identify whether the problem is data skew, excessive shuffling, too many small files, or insufficient resources. I would also check the Glue job metrics and logs in CloudWatch. Depending on the issue, I could improve partitioning, compact files, reduce unnecessary columns, or increase workers.

Remember: Find the bottleneck first and don't simply add more workers.

Q3. A Glue Crawler is creating multiple tables for data that should be one table. What would you check?

I would first check whether the S3 folder structure and schemas are consistent. Different file layouts or incompatible schemas can cause the crawler to create separate tables. I would use crawler grouping settings and exclude patterns where appropriate. For production pipelines, I may use the crawler only to bootstrap the catalog and then manage the schema and partitions explicitly.

Remember: Consistent S3 layout → consistent catalog tables.

Q4. Your Glue job successfully writes data to S3, but Athena cannot see the new partitions. What could be wrong?

The data may have been written to S3 without the corresponding partitions being registered in the Glue Data Catalog. I would check whether the job updates the catalog or whether partitions need to be added using ALTER TABLE ADD PARTITION, MSCK REPAIR TABLE, or another catalog-update mechanism. If using Iceberg, partition registration is managed differently because the table metadata tracks the data.

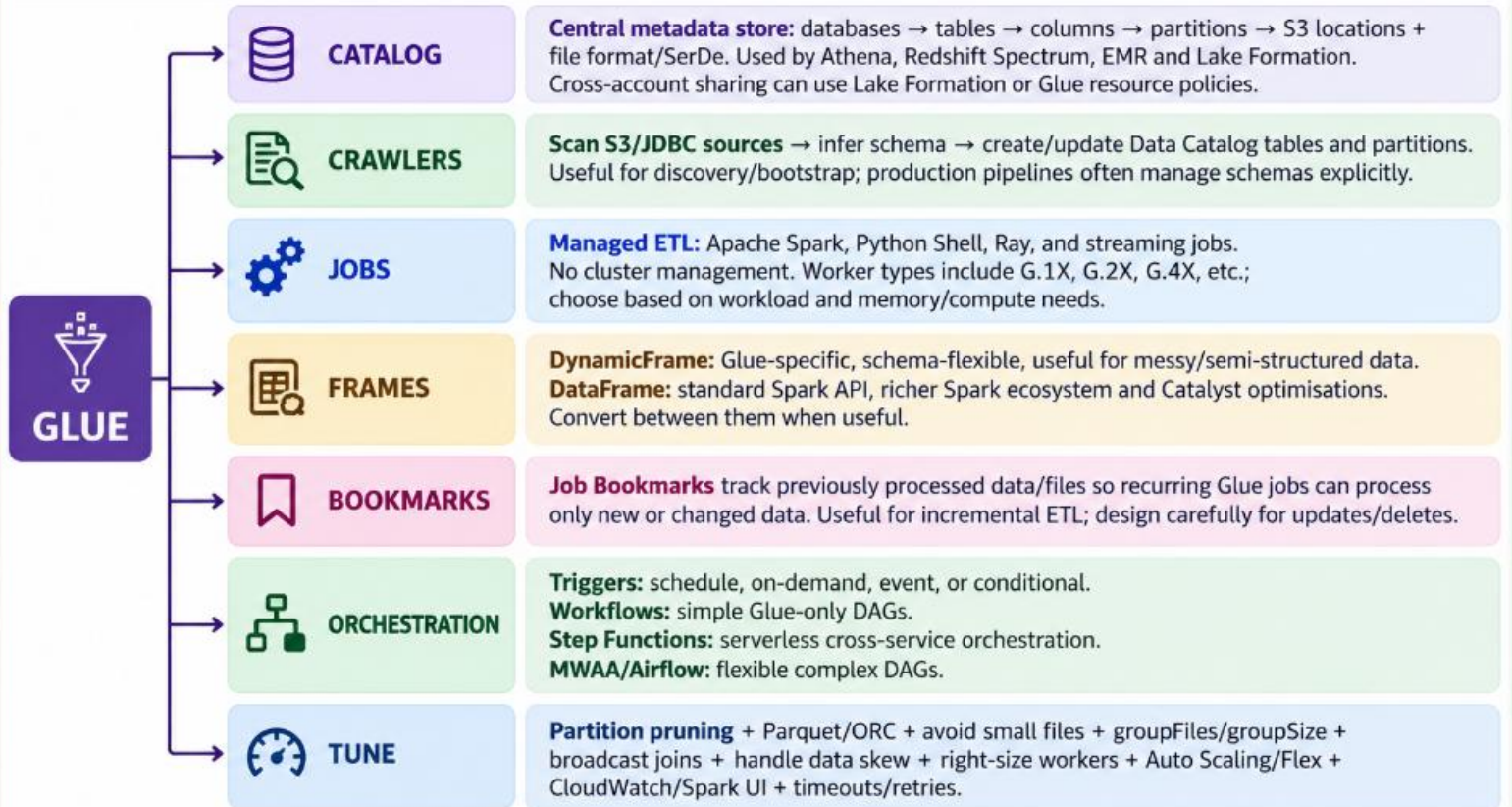
Remember: S3 contains the data; the Catalog tells Athena where and how to find it.

Q5. You need to build a pipeline with multiple Glue jobs, branching, retries, and calls to other AWS services. Would you use Glue Workflows?

For a simple Glue-only pipeline, I would use Glue Workflows because they are managed and easy to operate. For complex workflows involving branching, retries, and multiple AWS services, I would choose Step Functions. If the organisation already uses Airflow and needs highly flexible DAG orchestration, MWAA would also be a good option.

Remember: Glue Workflows = simple Glue pipelines; Step Functions = serverless cross-service orchestration; Airflow = maximum flexibility.

MENTAL MAP | GLUE = CATALOG + CRAWLERS + SERVERLESS SPARK



Key takeaway: Glue provides a managed, serverless Spark environment for ETL, with a central catalog, discovery via crawlers, and orchestration options for simple to complex pipelines.

YOUR NOTES

3 Amazon Redshift

MPP columnar warehouse: distribution decides shuffle, sort keys decide Input/Output

3.1 What it does and where it fits

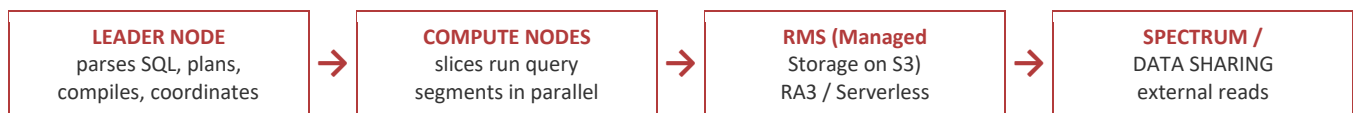
Amazon Redshift is AWS's cloud data warehouse, used to store and analyse large amounts of structured data using SQL. Data engineers typically load curated data from S3, databases, or other sources into Redshift tables, transform and join the data, and build fact and dimension tables for analytics and reporting. Redshift is designed for workloads with large datasets, complex joins, aggregations, and many users querying the data at the same time. In a typical pipeline, S3 acts as the data lake/storage layer, Glue or Spark performs data processing, and Redshift serves the final, business-ready data to tools such as Power BI and Tableau.

ANALOGY

Redshift is a warehouse with many aisles (nodes) and each aisle split into slices (workers). Distribution style decides which aisle a pallet goes to when it arrives. If two products that are always picked together sit in the same aisle, the picker never walks; if they sit in different aisles, every order means walking between aisles (that is the shuffle). Sort keys are the order the pallets are stacked within an aisle so the picker can skip whole stacks.

3.2 Key components

Architecture



- **Leader node.** Takes in queries, builds the plan, compiles the code, hands work to the compute nodes, and assembles the result. It holds no user data, and it's the only node clients talk to.
- **Compute nodes.** Each one is divided into slices, roughly one per vCPU. Every slice owns a share of each table's rows and runs its own part of every query step, so your parallelism is effectively the slice count.
- **Columnar storage.** Data is stored by column with per-column compression (AZ64, LZ0, ZSTD, delta, run-length). A query reads only the columns it references. Each 1 MB block carries min/max zone maps, letting Redshift skip blocks that can't match the predicate.
- **Storage tiers.** RA3 and Serverless cache hot data on local SSD while the full dataset lives in Redshift Managed Storage on S3, so compute and storage scale independently. Older DC2 nodes have local storage only.

- **Workload Management (WLM):** Queues with allocated memory and concurrency limits. Auto WLM with query priorities is the default and handles most cases.
- **Concurrency Scaling** Spins up transient clusters to absorb bursts, now for writes as well as reads.
- **Materialized views** the main acceleration tool, with auto-refresh and automatic query rewrite so existing queries benefit without being changed.
- **Result cache** the leader returns an identical query instantly when the underlying tables haven't changed.

Distribution styles

Style	Rows go to	Choose when	Risk
KEY	Rows are distributed across nodes based on a hash of the chosen DISTKEY column.	Large fact tables that are frequently joined on the same column. Using the same distribution key on both tables can keep matching rows together and reduce data redistribution.	Data skew if the key has low cardinality or uneven values. Choosing the wrong key can make joins slower.
ALL	A full copy of the table is stored on every compute node.	Small, relatively static dimension tables that are frequently joined with large tables.	Higher storage and load/refresh cost because the table is copied to every node
EVEN	Rows are distributed round-robin across the compute nodes.	No suitable distribution key exists, or the table is mainly scanned rather than joined.	Joins may require data redistribution , which can increase query time.
AUTO (default)	Redshift initially chooses a suitable distribution style and can change it automatically as the table grows and usage patterns change.	Good default when you don't yet know the best distribution strategy.	Automatic optimisation can take time; for critical workloads, monitor the table and query performance rather than assuming AUTO is always optimal.

Sort keys

Type	Behaviour	Use for
Compound (default)	Sorts data by the first column, then the second column within the first, and so on. The leading columns are the most important for query filtering.	Best for tables commonly filtered or joined using the leading sort-key columns, such as date-based queries
Interleaved	Gives the sort-key columns more equal importance, rather than prioritising the first column.	Useful when queries filter on different combinations of columns and there is no clear leading column. It is less commonly recommended today because it requires more maintenance.
AUTO	Redshift automatically chooses and can adjust the sort key based on workload patterns using Automatic Table Optimization.	Good default when you are not sure which sort key to choose.

Rule of thumb: sort key = the column most often in WHERE range predicates (usually event date), and if a table is joined on its DISTKEY, making that column also a sort key enables a **merge join**, the fastest join Redshift has.

COPY and UNLOAD

- **COPY** is the main high-performance way to load large amounts of data into Redshift. It loads data in parallel, commonly from S3. For efficient loading, use multiple reasonably sized files rather than one huge file, and use formats such as Parquet/ORC or compressed CSV. Use a manifest when you need to specify the exact files to load and use an IAM role rather than storing AWS access keys. For reliable pipelines, it is common to load into a staging table and then MERGE or perform a controlled delete-and-insert into the target.
- **Auto-copy** (COPY JOB) can automatically detect new files arriving in an S3 location and load them into Redshift. Zero-ETL integrations can also continuously replicate supported source databases into Redshift with minimal pipeline management. Mention these when asked about automated data ingestion.
- **UNLOAD** does the opposite of COPY: it exports query results from Redshift to S3, in parallel. You can write the output as Parquet or CSV and optionally partition it. It is useful for sharing data with Athena, feeding a data lake, or exporting/archive workloads.
- Avoid thousands of single-row INSERTs. For large or regular loads, use COPY or other bulk-loading approaches. Multi-row INSERT is reasonable for small amounts of data.

VACUUM and ANALYZE

When you update or delete rows in Redshift, the old rows can become ghost rows, and new data can be added to an unsorted region.

- VACUUM cleans up deleted space and re-sorts the table. You can use VACUUM DELETE ONLY to reclaim space, VACUUM SORT ONLY to improve sort order, or a full VACUUM for both.
- ANALYZE updates table statistics so the query optimizer can choose better execution plans, such as join strategies and join order. Redshift can perform these maintenance tasks automatically in the background.

Remember:

VACUUM = clean + sort

ANALYZE = refresh statistics

Query performance checklist

Symptom	Likely cause	Fix
DS_DIST_BOTH in plan	Both tables need to be redistributed before the join, often because the join columns do not align with the distribution strategy.	Review the distribution keys. For large tables, consider a suitable DISTKEY; for small dimension tables, consider DISTSTYLE ALL.
Full table scan despite a date filter	The date column is not a useful leading sort key, or the table has significant unsorted data.	Consider a date-based sort key, maintain sort order, and check whether the query is benefiting from zone-map pruning.
Query waits in queue	WLM resources are busy, often because long-running ETL workloads are competing with BI queries.	Review WLM queues/priorities, isolate workloads where appropriate, use concurrency scaling when suitable, and schedule heavy ETL during quieter periods.

Symptom	Likely cause	Fix
Dashboard query is repeatedly slow	The same expensive aggregations are being recalculated repeatedly.	Consider a materialized view for frequently used aggregations. Result caching may also help repeated identical queries when applicable.
Nested Loop join in the plan	Often caused by an inefficient join condition, such as a missing/equality join condition or an inequality join.	Review the join logic and check for an accidental CROSS JOIN or missing join condition.
Disk-based steps (is_diskbased)	A query step needs more memory than is available and spills intermediate data to disk.	Reduce intermediate data width, select only required columns, pre-aggregate data, or review workload/memory allocation.
stats_off is high	Table statistics are stale after significant data changes.	Run ANALYZE or allow automatic analyze to update the statistics.

Redshift Spectrum and the lakehouse boundary

Spectrum lets Redshift query S3 data registered in the Glue Data Catalog (external schema) using a separate fleet of Spectrum nodes, billed per TB scanned like Athena. Pattern: keep hot 12-24 months in Redshift, everything older as Parquet in S3, and expose one view that **UNION ALL**s both. Same tuning rules as Athena: Parquet, partition on filter columns, big files, push filters and aggregates to Spectrum, keep the join on the Redshift side small. Redshift also reads and writes Iceberg tables (including S3 Tables) via Lake Formation, and **data sharing** lets other clusters/accounts query live tables without copying. Redshift's SUPER type handles semi-structured JSON natively.

Saying you would put all history in Redshift "because it is faster". The senior answer talks about cost per TB in managed storage vs S3, and shows you know Spectrum, data sharing and zero-ETL exist so you are not copying data three times.

3.3 Scenario questions

Q1. A Redshift query joining a large fact table with a large dimension table is very slow. The EXPLAIN plan shows DS_DIST_BOTH. What would you do?

I would check the distribution style and distribution keys of both tables. DS_DIST_BOTH means both tables are being redistributed across nodes before the join, which is expensive. If the tables are frequently joined on the same column, I would consider using a suitable DISTKEY to co-locate the data. If one table is small enough, I would consider DISTSTYLE ALL.

Remember: DS_DIST_BOTH → data movement → review distribution.

Q2. A Redshift table has grown significantly and queries using the date column are becoming slower. What would you investigate?

I would check whether the date column is an appropriate sort key and whether the table has become significantly unsorted. I would check SVV_TABLE_INFO, especially the unsorted percentage. If necessary, I would maintain the sort order with VACUUM SORT and review whether the queries are benefiting from zone-map pruning.

Remember: Good sort key → less data scanned.

Q3. Your BI queries are frequently waiting in the Redshift queue because a large ETL job is running. How would you solve this?

I would separate workloads so that heavy ETL processing does not unnecessarily block interactive BI queries. Depending on the environment, I could use WLM priorities/queues, concurrency scaling, or schedule heavy ETL during off-peak periods. I would also investigate whether Serverless is a better fit if the workload needs automatic scaling.

Remember: Separate workloads → protect BI performance.

Q4. A dashboard repeatedly calculates the same large aggregation and is taking several seconds every time. What would you do?

I would consider creating a materialized view containing the commonly used aggregation so the dashboard does not repeatedly perform the expensive calculation. I would also check whether result caching can help for identical queries. The choice depends on how frequently the underlying data changes and how fresh the dashboard needs to be.

Remember: Repeated expensive aggregation → consider a materialized view.

Q5. A Redshift ETL query suddenly becomes much slower after a large data load. How would you troubleshoot it?

I would check the query plan and Redshift system information to identify the bottleneck. I would look for stale statistics, unsorted data, data skew, redistribution such as DS_DIST_BOTH, and disk-based steps. I would run ANALYZE if statistics are stale, maintain sort order if needed, and review distribution or query design if there is excessive data movement.

Remember: Don't immediately add compute-first identify whether the problem is statistics, sorting, skew, distribution, or memory.

Section	Key concepts
● REDSHIFT ARCHITECTURE	Provisioned or Serverless • Leader node parses/plans/coordinates • Compute nodes execute queries using slices • RA3 separates compute from storage using Redshift Managed Storage • Columnar storage + compression + zone maps
● DIST	KEY → co-locate large table joins • ALL → replicate small dimension tables to every node • EVEN → when there is no good distribution key • AUTO → Redshift chooses/adjusts automatically • DS_DIST_NONE = good • DS_DIST_BOTH = investigate
● SORT	COMPOUND, INTERLEAVED, AUTO • Helps with range filters, especially dates • Enables zone-map pruning • Check unsorted % and maintain sort order when needed
● IN / OUT	COPY → bulk-load data, commonly from S3 • Use multiple reasonably sized files, compression/columnar formats, manifests when needed, and an IAM role • Staging table → MERGE into target • UNLOAD → export results to S3 • Auto-copy → automatically load new S3 files

Section	Key concepts
● MAINTAIN	Updates/deletes can create ghost rows and new data can be unsorted • VACUUM → reclaims space and/or improves sort order • ANALYZE → updates table statistics • Check SVV_TABLE_INFO → unsorted, stats_off, skew_rows
● TUNE	Read the query plan: DS_DIST_NONE = good • DS_DIST_ALL_NONE = good • DS_BCAST_INNER = usually OK when small • DS_DIST_BOTH = investigate • Check data skew, disk-based steps, nested loops and WLM queue waits • Use materialized views for repeated expensive aggregations
● USE CASES	Data warehousing • BI and analytics • Large aggregations • Complex joins • High-concurrency dashboards • Integrates with S3, Glue, Athena, Redshift Spectrum and BI tools such as Power BI/Tableau

YOUR NOTES

4 Amazon Athena

Serverless SQL on S3.

4.1 What it does and where it fits

Amazon Athena is a serverless SQL query service that lets you query data directly in S3 without loading it into a database or data warehouse. Data engineers commonly use Athena for ad-hoc analysis, data exploration, validation, and querying data lake tables. Athena uses the Glue Data Catalog to understand the table schema, partitions, and S3 location, while the actual data remains in S3.

Typical data lake flow:

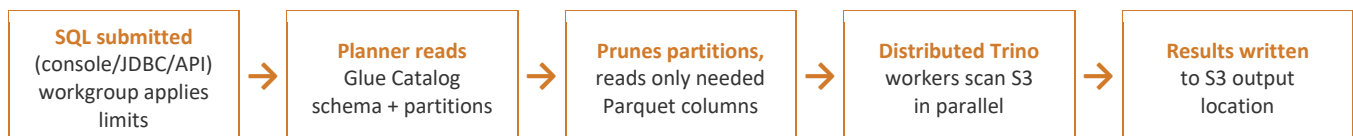
S3 → Glue Catalog → Athena → SQL analysis / BI

ANALOGY

Think of Athena as a contractor who charges based on how much data they read, not how long they work. If you give Athena well-organised Parquet files with partitions, it can read only the required columns and partitions, so you pay for much less data. If you give it large amounts of unpartitioned CSV or JSON, it may need to scan much more data, making queries slower and more expensive.

4.2 Key components

Architecture and how a query runs



- **Workgroups** help separate teams and control usage. Each workgroup can have its own S3 query-result location, data-scan limits, query settings, tags, and CloudWatch metrics. Use data-scan limits to prevent unexpectedly expensive queries.
- **Athena engine** versions provide the SQL query engine used by Athena. Engine version 3 is based on Trino and provides improved SQL capabilities and performance, including features for joins and data filtering. Athena for Apache Spark is a separate service for running Spark applications.
- **Capacity Reservations** provide dedicated Athena processing capacity for workloads that need predictable performance and concurrency, rather than relying entirely on on-demand capacity.
- **Federated Query** allows Athena to query data from sources such as RDS, DynamoDB, Redshift, and CloudWatch Logs using Lambda-based connectors. The advantage is querying multiple sources with SQL; the trade-off is additional latency and Lambda costs.
- **Query results** are stored in S3, so you need an S3 output location and incur normal S3 storage costs. Query result reuse can return a previous result for eligible repeated queries within a configured maximum age, reducing unnecessary query processing and cost.

Remember:

Workgroups → control & cost

Engine → query performance & features

Capacity → predictable concurrency

Federated Query → query external sources

Result reuse → avoid repeated processing

Partitioning and partition projection

Athena uses the Glue Data Catalog to identify which partitions need to be scanned. With Hive-style partitions such as year=2026/month=09/day=14, the partitions normally need to be registered using a crawler, MSCK REPAIR TABLE, ALTER TABLE ADD PARTITION, or the Glue API. At large scale, this can become difficult to manage, and having hundreds of thousands of partitions can also increase query planning time. Partition projection solves this by defining the partition values and S3 path pattern in the table properties, so Athena can calculate the required S3 paths without storing every partition in the Glue Catalog. It works particularly well for time-series data with predictable partition paths. Another option is Iceberg hidden partitioning, where Iceberg manages partition metadata and transforms such as days(timestamp) or bucket(16, id), allowing users to filter on the original columns while Athena still performs partition pruning.

Remember:

Traditional partitions → Catalog tells Athena where to look.

Partition projection → Athena calculates where to look.

Iceberg → Table metadata manages the partitions.

Parquet and ORC

	Parquet	ORC
Ecosystem	Widely used across AWS Glue, Spark, Athena, and Redshift UNLOAD	Strong Hive/Presto ecosystem and commonly used in Hive-based environments

	Parquet	ORC
Structure	Data is organised into row groups → column chunks → pages . The file footer stores schema and statistics such as min/max values.	Data is organised into stripes → column streams , with indexes and optional bloom filters for efficient filtering.
Compression	Supports Snappy, ZSTD, GZIP , and other codecs. Snappy is commonly chosen for speed; ZSTD provides better compression.	Commonly uses ZLIB, Snappy, and ZSTD , depending on the engine and configuration.
Best Choice	Parquet is usually the default choice on AWS , especially for new data lake projects using Spark/Athena/Glue.	Good choice when an existing platform or organisation already standardises on ORC.

Why Columnar Storage Matters

Columnar formats such as Parquet store data by column rather than by row. If a table has 60 columns but a query needs only 4, the query engine can read mainly those 4 columns instead of scanning all 60. Parquet also stores statistics such as min/max values for groups of rows, allowing Athena to skip data that cannot match the query filter. For good performance, aim for reasonably sized files, roughly 128 MB–1 GB, and avoid creating thousands of small files.

Remember:

Column pruning → read fewer columns.

Predicate pushdown → skip unnecessary

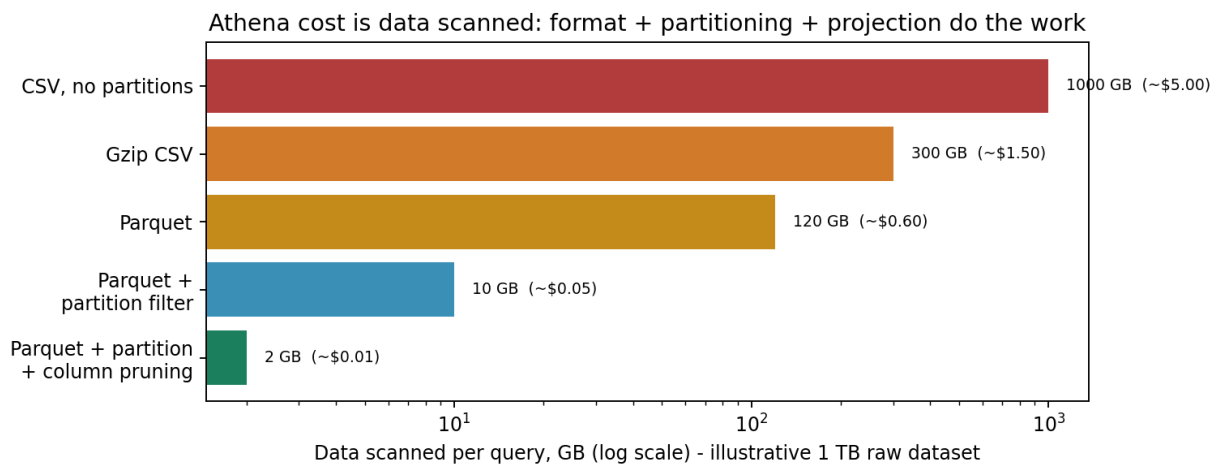


Figure 2. Illustrative effect of format and layout on bytes scanned (and therefore cost) for the same logical query.

CTAS and INSERT INTO

- **CREATE TABLE AS SELECT (CTAS)** creates a new table and writes the query results to S3 in the format you choose, such as Parquet with ZSTD compression. You can also partition the output. It is a simple and cost-effective way to convert JSON/CSV data into partitioned Parquet without running a Spark job.

A CTAS statement has a 100-partition limit, so larger loads may need to be split across multiple statements.

- INSERT INTO adds new data to an existing table, usually by creating additional files in S3. It is useful for incremental daily loads into a Parquet table. It does not automatically remove duplicates, so your pipeline needs to handle idempotency if the same data can be loaded more than once.
- Athena Iceberg tables support operations such as INSERT, UPDATE, DELETE, and MERGE. Athena also supports OPTIMIZE for compaction and VACUUM for removing old snapshots/files according to the table's retention settings. For workloads with frequent updates and deletes, regular maintenance is important.
- UNLOAD exports the results of a query from Athena to S3 without creating a table. You can write the results in formats such as Parquet, JSON, or text-based formats, making it useful for sharing query results or feeding another data pipeline.

Remember:

CTAS → create a new table

INSERT INTO → append data

Iceberg → update/delete/merge + table maintenance

UNLOAD → export query results to S3

Athena and the Glue Data Catalog

Athena uses the AWS Glue Data Catalog to store metadata about databases, tables, schemas, partitions, and S3 locations—it does not maintain a separate metastore. This means a table created in Athena is also available to services such as Glue, Redshift Spectrum, and EMR through the shared catalog. Lake Formation can provide fine-grained access control, such as table, column, and row-level permissions. When you run DDL in Athena, such as ALTER TABLE ADD COLUMNS, you are updating the table metadata in the Glue Catalog. For Parquet tables, schema changes can generally be handled by matching columns by name rather than position, but column renames require additional care because the old and new column names are treated as different fields.

Remember:

S3 = data → Glue Catalog = metadata → Athena = query.

Cost and query optimisation

Lever	Action	Typical effect
Format	Use Parquet or ORC with Snappy/ZSTD instead of CSV/JSON.	Reads less data → lower cost and faster queries.
Partitioning	Partition by commonly filtered columns, usually date. Use partition projection when appropriate.	Can dramatically reduce the amount of data scanned.
File size	Compact small files into reasonably sized files, typically 128 MB–1 GB.	Fewer files → less overhead and better query performance.
Column pruning	Select only the columns you need; avoid SELECT *.	Athena reads fewer columns → less data scanned.

Lever	Action	Typical effect
Filtering	Filter on partition columns and other selective columns as early as possible.	Reduces the amount of data that needs to be processed.
Joins	Reduce the data before joining; select only required columns and filter both sides where possible.	Less data to process → lower memory use and faster joins.
Approximate functions	Use functions such as <code>approx_distinct()</code> instead of <code>COUNT(DISTINCT)</code> when exact results are not required.	Can use less memory and processing time.
Result reuse	Enable query result reuse with an appropriate maximum age for repeated queries, such as dashboards.	Avoids reprocessing the same eligible queries → lower cost and faster response.
Guardrails	Set per-query and workgroup data-scan limits and monitor Athena metrics in CloudWatch.	Helps prevent unexpected query costs.
Capacity	Consider capacity reservations when predictable concurrency and performance are more important than purely on-demand pricing.	More predictable performance and capacity.

4.3 Scenario questions

Q1. An Athena query scans 20 TB of data even though the user only needs data from the last 7 days. What would you do?

I would check whether the table is partitioned by date and whether the query includes a filter on the partition column. I would also check whether the data is stored in Parquet/ORC and whether the query is selecting unnecessary columns. If partitions are properly designed and the query filters them, Athena can scan only the required data.

Remember: Partition pruning + column pruning = less data scanned.

Q2. Your Athena table contains millions of small Parquet files and queries are slow. How would you fix it?

I would introduce a compaction process to combine small files into reasonably sized files, typically around 128 MB–1 GB. I would also review the partitioning strategy because excessive partitioning can create too many small files. The goal is to have fewer, larger files that Athena can process efficiently.

Remember: Too many small files → more overhead → slower queries.

Q3. Your company has a time-series S3 dataset with hundreds of thousands of partitions. Managing partitions in the Glue Catalog has become difficult. What would you use?

I would consider Athena Partition Projection if the partition values and S3 paths follow a predictable pattern, such as daily dates. Instead of registering every partition in the Glue Catalog, Athena calculates the required S3 paths from the table properties. If the data uses Iceberg, I would consider Iceberg partitioning, where the table metadata manages the partition information.

Remember: Predictable partitions → Partition Projection.

Q4. Several teams run Athena queries, and occasionally someone runs a query that scans several terabytes and creates a large, unexpected bill. How would you prevent this?

I would use Athena Workgroups to separate workloads and configure data-scan limits at the query/workgroup level. I would also monitor Athena metrics through CloudWatch and create alarms for unusually high data scanned. For frequently repeated dashboard queries, I would consider query result reuse where appropriate.

Remember: Workgroups + scan limits + monitoring = cost control.

Q5. A business team wants to query data from S3 and an RDS database using one SQL query. They don't want to build a separate ETL pipeline. How would you approach it?

I would consider Athena Federated Query, which uses Lambda-based connectors to query supported external data sources such as RDS and DynamoDB alongside S3 data. It is useful for ad-hoc or occasional cross-source queries, but I would consider ETL into a data lake or warehouse for large, frequent workloads because federated queries can introduce additional latency and processing costs.

Remember: Federated Query → query multiple data sources with SQL without first moving everything into S3.

MENTAL MAP | ATHENA (TRINO) SQL – PAY FOR BYTES SCANNED

SELECT	Pick only the columns you need (column pruning) – avoid SELECT * to reduce data scanned. Example: <code>SELECT customer_id, order_date, amount FROM sales;</code>
FROM	Reads data from S3 via the Glue Catalog (external tables). Make sure the table is in the right database and location. Example: <code>FROM analytics.sales;</code>
WHERE	Filter early, especially on partition columns (partition pruning) to reduce the amount of data scanned. Example: <code>WHERE dt = '2025-09-14' AND country = 'AU';</code>
GROUP BY	Used for aggregations. Select only the columns needed for grouping. Example: <code>SELECT country, COUNT(*) FROM sales WHERE dt = '2025-09-14' GROUP BY country;</code>
HAVING	Filter aggregated results (used with GROUP BY). Example: <code>HAVING COUNT(*) > 100;</code>
ORDER BY	Sorts the final result set. Expensive for large datasets as it runs on a single worker – use with LIMIT. Example: <code>ORDER BY order_date DESC LIMIT 100;</code>
LIMIT	Restricts the number of rows returned. Useful for testing and exploration to save costs. Example: <code>LIMIT 100;</code>

5 AWS IAM

Who can do what, to which resource, under which conditions

5.1 What it does and where it fits

AWS IAM (Identity and Access Management) is the authorization layer for AWS. It controls who or what can access which AWS resources and what actions they can perform. IAM is not a separate step in a data pipeline; it applies across the entire pipeline. For example, when a Glue job reads data from S3, the Glue service assumes an IAM role, and AWS evaluates that role's permissions along with the S3 bucket policy, KMS key policy, and, where applicable, Lake Formation permissions. As a senior data engineer, you should understand how to design these roles and permissions using least privilege, rather than simply relying on a platform team to provide access.

ANALOGY

Think of IAM as a **building's access-control system**. A **user** is an employee with their own identity and permissions. A **role** is a temporary access badge that an AWS service or trusted user can **assume** when needed; the badge provides temporary credentials and is not permanently assigned to them. **Policies** define which doors and actions that identity or role is allowed to use. A **resource-based policy** is like a rule attached directly to a door, specifying who can access it. This is especially useful for **cross-account access**, where you want an identity from another AWS account to access your resource.

5.2 Key components

Principals: users, groups, roles

Principal	What	Data-engineering use
User	A long-term AWS identity that can have a password and/or access keys.	Avoid for applications and pipelines. Humans should generally use IAM Identity Center (SSO) and temporary credentials instead of long-lived access keys.
Group	A collection of IAM users that share the same policies.	Useful for managing permissions for groups of users, although modern environments often use IAM Identity Center instead.
Role	An identity that does not have permanent credentials. It is assumed to obtain temporary credentials.	The standard choice for Glue, Lambda, EC2, Redshift, and other workloads. Also commonly used for cross-account access and for humans accessing AWS through federation.
Service-linked role	A special IAM role that is created and managed by an AWS service for actions the service needs to perform on your behalf.	Usually created automatically. Important to recognise when troubleshooting service permissions or configuration issues.

Principal	What	Data-engineering use
Root	The identity with complete access to the AWS account.	Never use for day-to-day work. Secure it with MFA and keep its use limited to tasks that specifically require root access.

Policies: the JSON and the evaluation

An IAM policy is a JSON document containing statements that define what an identity or service is allowed to do. A statement typically includes Effect (Allow or Deny), Action (such as `s3:GetObject`), and Resource (the AWS resource ARN). You can also use Condition to make permissions more specific, and resource-based policies can include a principal to specify who is allowed to access the resource.

The key rule: explicit Deny always wins. If there is no explicit Deny, AWS evaluates the applicable policies and the request must have the required Allow permissions, including any applicable SCPs, permission boundaries, or session policies. If nothing allows the request, the result is an implicit Deny.

For same-account access, an Allow can generally come from either an identity-based policy or a resource-based policy. For cross-account access, both accounts must participate in granting access-the trusted resource/account must allow the principal, and the principal must have permission to perform the action.

Remember:

- Explicit Deny → always wins
- No Allow → implicit Deny
- Same account → identity or resource policy can allow
- Cross account → permissions must be correctly allowed on both sides

Policy type	Attached to	Typical statement	Notes
Identity-based	User, group, role	"This role may <code>glue:GetTable</code> on <code>arn:aws:glue:*:*:table/db/*</code> "	Managed (AWS or customer) or inline
Resource-based	S3 bucket, KMS key, Glue catalog, Lambda, SQS, secrets	"Role X in account B may <code>s3:GetObject</code> here"	The only way to grant cross-account without a role switch; has a Principal element
Trust policy	Role (a special resource policy)	" <code>glue.amazonaws.com</code> may <code>sts:AssumeRole</code> this role"	Defines who can assume; add <code>aws:SourceAccount</code> conditions to avoid confused deputy
Permissions boundary	User or role	Maximum permissions ceiling	Lets teams create roles without escalating beyond the boundary
SCP	Org unit / account	"No one in this OU may disable CloudTrail"	Guardrail, never grants
Session policy	Passed at AssumeRole	Scopes down a session further	Used by brokers and data-access services

AssumeRole and cross-account access



When a workload in Account A needs to access resources in Account B, one common approach is IAM role assumption. The role in Account A needs permission to call `sts:AssumeRole`, and the target role in Account B needs a trust policy that allows Account A's role to assume it. AWS STS then provides temporary credentials, and the workload uses those credentials with the permissions attached to the role in Account B. For third-party access, an `ExternalId` can help prevent the confused-deputy problem. For organisations, `aws:PrincipalOrgID` can restrict access to principals belonging to your AWS Organization.

Another approach is a resource-based policy, such as an S3 bucket policy, where Account B directly grants Account A's role access to the resource. This can be simpler for S3 data sharing because the caller does not need to switch roles. For cross-account S3 access, make sure the required S3 permissions and KMS permissions are also correctly configured when SSE-KMS is used.

Least privilege in practice

- Use specific resources: Scope permissions to the required S3 bucket and prefixes instead of using Resource: "*". Separate read and write permissions where practical.
- Use one role per workload: Give each workload its own role, such as `glue-silver-orders-role`, rather than creating one large role with permissions for an entire team. This makes permissions easier to audit and reduces the blast radius of a compromise.
- Use IAM Access Analyzer: Use it to identify overly broad or unused access and to help generate policies based on actual CloudTrail activity. Review permissions regularly and remove unnecessary access.
- Use Lake Formation for data governance: For analytics environments, Lake Formation can provide database, table, column, and row-level permissions on data registered in the Glue Data Catalog. This is often cleaner than managing many S3 prefix-based policies.
- Protect secrets: Store database passwords, API keys, and other secrets in AWS Secrets Manager rather than putting them directly into job parameters, source code, or plain-text environment variables. Enable rotation where appropriate.
- Enforce encryption and secure transport: Use S3 bucket policies to require TLS/HTTPS and, where required by your security standard, require server-side encryption for uploads. With SSE-KMS, ensure the workload also has the necessary KMS permissions.

What each service needs

Service	Trust policy principal	Permissions the role needs	Frequent misses
Policy type	Attached to	What it does	Typical use
Identity-based policy	User, group, or role	Defines what actions the identity is allowed or denied on specific AWS resources.	Example: A Glue role can <code>glue:GetTable</code> on specific Glue Catalog tables.

Service	Trust policy principal	Permissions the role needs	Frequent misses
Resource-based policy	AWS resource, such as S3 bucket, KMS key, SQS queue, Lambda function	Defines who can access the resource and what actions they can perform. Uses a <code>Principal</code> .	Common for cross-account access, such as allowing a role in Account A to read objects in an S3 bucket in Account B.
Trust policy	IAM role	Defines who or what is allowed to assume the role using <code>sts:AssumeRole</code> .	Example: Allowing <code>glue.amazonaws.com</code> to assume a Glue execution role.
Permissions boundary	User or role	Sets the maximum permissions that the identity can receive. It does not grant permissions by itself.	Useful when allowing teams to create roles while preventing them from granting permissions beyond an approved boundary.

5.3 Scenario questions

Q1. Your Glue job can read from S3, but it fails with `AccessDenied` when reading SSE-KMS encrypted files. How would you troubleshoot it?

I would check the Glue job's IAM role first to make sure it has the required `s3:GetObject` permission. Then I would check the KMS key policy and the role's permissions to make sure it can use the key, including `kms:Decrypt`. I would also check for an explicit `Deny` in the bucket policy, KMS policy, SCP, or permissions boundary.

Remember: S3 access + KMS access + no explicit `Deny`.

Q2. A team asks for `AdministratorAccess` because their Glue jobs keep getting permission errors. How would you handle it?

I would not give `AdministratorAccess` immediately. I would identify exactly which AWS resources and actions the Glue jobs need and create a dedicated IAM role using least privilege. I would use CloudTrail and IAM Access Analyzer to understand the permissions actually required, then grant only those permissions.

Remember: Fix the missing permission, not the symptom with admin access.

Q3. A Glue job in Account A needs to read data from an S3 bucket in Account B. How would you design the access?

I would first decide whether Account A should access the bucket directly or assume a role in Account B. For direct S3 access, Account B can use a bucket policy to allow Account A's role to read the required objects. If the workload needs to operate as an identity in Account B, I would use cross-account `AssumeRole`. If the data uses SSE-KMS, I would also configure the KMS permissions correctly.

Remember: Cross-account → S3 permissions + KMS permissions if encrypted.

Q4. Developers need to create IAM roles for their applications, but security does not want them creating overly powerful roles. What would you use?

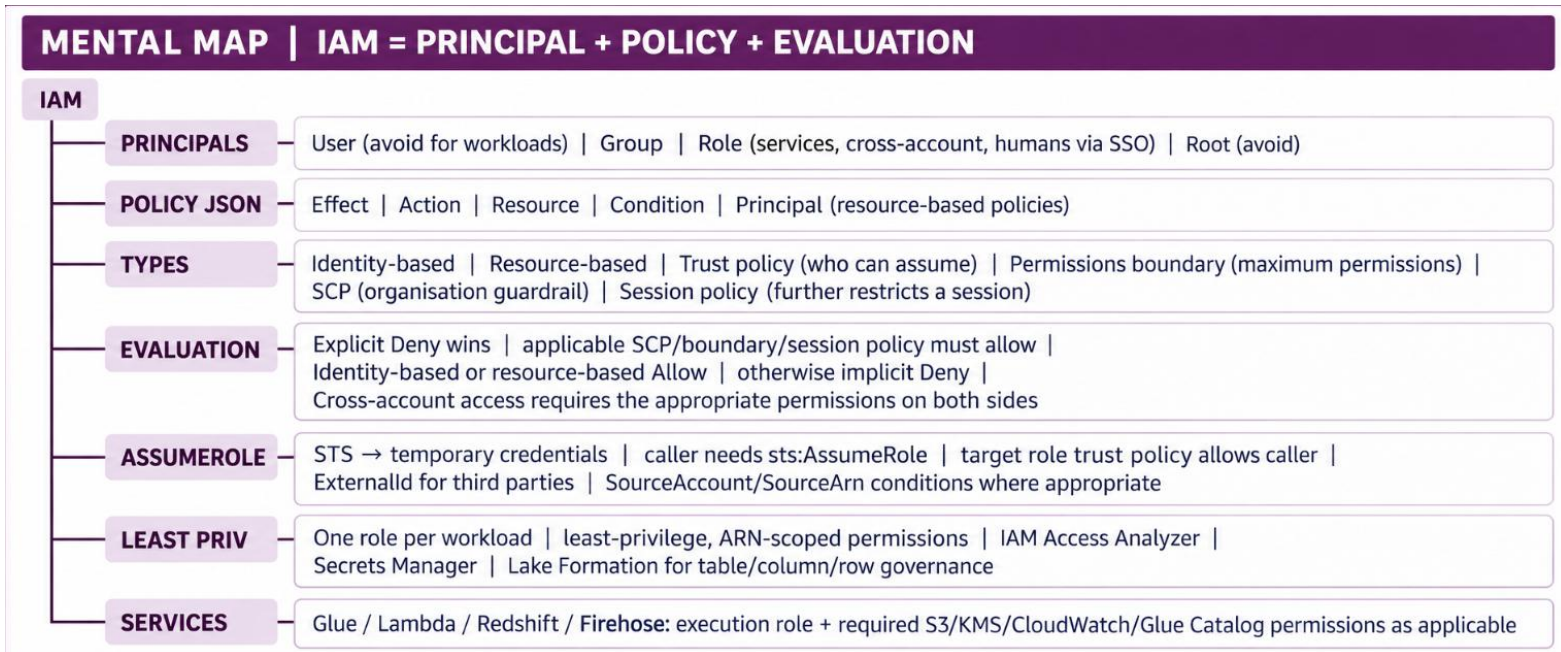
I would use a permissions boundary. The boundary defines the maximum permissions the developers' roles can have. Developers can create or manage roles within that limit, but they cannot grant permissions beyond the boundary. I would also use SCPs as organisation-level guardrails where appropriate.

Remember: Permissions boundary = maximum permissions for a role/user.

Q5. Your company has hundreds of AWS accounts and wants to ensure nobody can disable CloudTrail. How would you enforce this?

I would use an AWS Organizations SCP as a central guardrail to deny actions that could disable or weaken CloudTrail across the relevant accounts or OUs. The SCP does not grant CloudTrail permissions; it limits what accounts can do even if their IAM policies otherwise allow the action.

Remember: IAM controls permissions within an account; SCPs provide organisation-level guardrails.



YOUR NOTES

6 Amazon Kinesis

A durable, ordered, replayable event log. Shards decide throughput and ordering.

6.1 What it does and where it fits

Amazon Kinesis is used to ingest and process real-time streaming data. Kinesis Data Streams (KDS) is a managed, partitioned streaming service where producers continuously write records and multiple consumers can read the same stream independently. It is commonly used for clickstream data, IoT events, application logs, and CDC events and acts as the buffer between data producers and downstream processing. Amazon Data Firehose is a managed delivery service that automatically buffers and delivers streaming data to destinations such as S3, Redshift, OpenSearch, and supported third-party services, with optional transformations and format conversion. Managed Service for Apache Flink is used when you need more advanced real-time processing such as windows, joins, aggregations, and stateful processing.

Simple flow:

Applications / IoT / CDC → Kinesis Data Streams → Flink / Lambda / consumers → S3 / Redshift / other destinations

- **Remember:**

KDS → ingest and buffer streams

Firehose → deliver streams

Flink → process streams

ANALOGY

Think of a Kinesis Data Stream as a set of conveyor belts (shards). Producers place records on the belts, and records are retained for a configurable period so consumers can read them later and process them independently. The partition key determines which belt a record goes to, so records with the same partition key are kept in order. Firehose is different: it collects incoming records, buffers them, and automatically delivers them to a destination such as S3 or Redshift when the buffer fills or the configured time interval is reached. Firehose is designed for delivery rather than replaying records from arbitrary positions like Kinesis Data Streams.

6.2 Key components

Shards, partition keys, capacity modes

Concept	Numbers to know	What it means for design
Shard	In provisioned mode, each shard supports up to 1 MB/s or 1,000 records/s for writes and 2 MB/s for reads. The default maximum record size is 1 MiB, with support for records up to 10 MiB when large-record support is enabled. (AWS Documentation)	In provisioned mode, shard count determines your baseline throughput. For example, 50 MB/s of sustained writes requires at least 50 shards.
Partition key	The partition key determines which shard receives a record. Records with the same partition key are routed to the same shard and maintain ordering within that shard.	Use a high-cardinality, well-distributed key such as user ID or device ID. A hot key can overload one shard even when the overall stream has capacity.
Sequence number	Each record receives a unique sequence number within its shard.	Consumers can use sequence numbers/checkpoints to track progress and resume processing after failures.
Provisioned mode	You choose the number of shards and can scale the shard count as required.	Good for predictable, steady workloads where you can plan capacity and want direct control over shards.
On-demand mode	AWS automatically manages capacity based on traffic. New streams start with 4 MB/s write and 8 MB/s read throughput; regional scaling limits are much higher. (AWS Documentation)	Good for variable or unpredictable workloads because you don't need to manually provision shards.
Enhanced Fan-Out	Each registered EFO consumer can receive up to 2 MB/s per shard of dedicated read throughput. On-demand Advantage supports up to 50 registered EFO consumers; standard on-demand and provisioned support up to 20. (AWS Documentation)	Use EFO when multiple consumers need low-latency reads without competing for the shared 2 MB/s shard read capacity.

Producers

- PutRecord / PutRecords
 - PutRecord sends one record to a Kinesis stream.
 - PutRecords sends records in batches - up to 500 records or 5 MB.
- Kinesis Producer Library (KPL)
 - Batches and aggregates many small records into larger Kinesis records.
 - Especially useful for high-volume events smaller than 1 KB.
 - Provides retries, batching, aggregation and metrics.
- Kinesis Agent
 - Continuously tails log files and sends new log data to Kinesis.
- Native AWS producers
 - CloudWatch Logs → subscription filters can send logs to Kinesis.
 - EventBridge → routes events to Kinesis.
 - IoT Core → sends device/IoT events.
 - DMS → streams CDC changes into Kinesis.
 - Firehose → can consume data from Kinesis Data Streams and deliver it to destinations.
- Handling throttling
 - ProvisionedThroughputExceededException means the stream/shard is receiving more traffic than its available throughput.
 - Use exponential backoff with jitter for retries.
 - Check your partition key to make sure data is distributed evenly.
- Partition key is critical
 - The partition key determines which shard receives the record.
 - A good key distributes records across shards.
 - A poor key can create a hot shard.
- Example
 - customer_id → generally a good partition key because many customers distribute the traffic.
 - timestamp → can be problematic if many records arrive with the same timestamp/key and concentrate traffic.

REMEMBER: Producer → Partition Key → Shard → Throughput.

Bad partition key = hot shard = throttling.

Consumers

Consumer	Read model	Use when
Lambda – Event Source Mapping	Lambda polls each shard and invokes your function with batches. Supports batch size, batching window, parallelization factor (up to 10 batches/shard), starting position, bisect on error, retries, maximum record age, and on-failure destinations.	Default choice for simple event-driven processing with minimal operational overhead.
Kinesis Client Library (KCL)	Long-running application on EC2/ECS/EKS. Uses a DynamoDB lease table to track shard ownership and checkpoints.	Custom consumers that need more control over processing and shard management.
Enhanced Fan-Out (EFO)	Gives each registered consumer dedicated 2 MB/s per shard and uses HTTP/2 <code>SubscribeToShard</code> for low-latency delivery.	Multiple consumers or applications requiring low-latency reads without competing for shared read throughput.
Glue Streaming / Managed Service for Apache Flink	Glue Streaming uses Spark Structured Streaming for continuous/micro-batch ETL. Managed Flink supports stateful processing, windows, joins, and aggregations.	Streaming ETL and complex/stateful stream processing.

Retention and replay

- Consumers can choose where to start reading:
 - LATEST → start from new records arriving now.
 - TRIM_HORIZON → start from the oldest record still available.
 - AT_TIMESTAMP → start from a specific time.
 - Sequence number → start from a specific record position.
- Replay is possible within the Kinesis retention period.
 - Re-run a consumer after fixing a bug.
 - Backfill a new consumer.
 - Reprocess events into a rebuilt table.
- Kinesis Data Streams is therefore a replayable buffer, while Firehose is primarily a delivery service.
- Firehose can retry failed deliveries and send failed records to a backup/error location, but it is not designed as a general-purpose replay mechanism for successfully delivered records.
- Common architecture:
 - Applications → Kinesis Data Streams → Firehose → S3
 - Kinesis provides buffering + replay.
 - Firehose provides managed delivery.
 - S3 becomes the permanent historical/replay source.

Scaling and resharding

- UpdateShardCount
 - Changes the number of shards for the stream.
 - Useful when you need to scale overall stream capacity.
 - Current AWS limits and scaling behavior should be checked for the stream's mode/region.
- SplitShard / MergeShard
 - Historically used to target specific hot or cold shards.
 - Splitting creates child shards and increases capacity for that portion of the key space.
 - Merging combines adjacent shards when capacity is no longer needed.
- Ordering during resharding
 - Parent shards close after accepting writes.
 - Consumers continue reading the remaining records from the parent before moving to child shards.
 - Ordering for records with the same partition key is maintained.
- Important CloudWatch metrics to monitor:
 - WriteProvisionedThroughputExceeded → producers are being throttled.
 - ReadProvisionedThroughputExceeded → consumers are exceeding shared read throughput.
 - IncomingBytes / IncomingRecords → shows incoming traffic.
 - GetRecords.IteratorAgeMilliseconds → shows how far behind the consumer is.
- Iterator age is a critical streaming-health metric.
 - Increasing iterator age = consumer is falling behind.
 - A sustained increase should trigger investigation.
- On-Demand mode
 - AWS automatically manages shard capacity, reducing the need for manual resharding.

- However, it does not eliminate hot-key problems.
- A single partition key still maps to a single shard, so that key can become a bottleneck.

REMEMBER: Retention = replay window · Resharding = capacity management · Iterator age = consumer lag · On-Demand = less capacity management, not a solution for hot keys.

Firehose specifics

Setting	Detail	Interview angle
Buffering	Buffer size and interval control when Firehose delivers data. Delivery happens when the configured size or time threshold is reached.	Firehose provides near-real-time delivery, but it is not designed for millisecond-level streaming.
Transformation	Can invoke Lambda to transform records before delivery. It also supports data format conversion such as JSON → Parquet/ORC using Glue Data Catalog schemas.	Convert data before landing so the data lake is already in an analytics-friendly columnar format.
Dynamic Partitioning	Can extract partition keys from records and write to S3 prefixes such as <code>year=2026/month=09/day=14/</code> .	Creates an Athena-friendly S3 layout without requiring a separate partitioning job.
Destinations	Supports destinations including S3, Redshift, OpenSearch, Iceberg tables, Splunk, Datadog, and HTTP endpoints.	Choose Firehose when you need managed streaming delivery rather than custom stream processing.
Error Handling	Firehose retries failed deliveries and can send failed records to an S3 backup/error location. CloudWatch metrics can be used to monitor delivery success and failures.	Understand the difference between delivery retry/error handling and replay. Successfully delivered records aren't retained by Firehose for arbitrary replay.

The decision spine an interviewer wants: Firehose when data only has to land; Data Streams when something has to process it or several things need to read it; Flink when the processing is stateful. Then defend the shard math and the partition key.

6.3 Scenario questions

Q1. Clickstream generates 30,000 events/sec, average 800 bytes, with three consumers: fraud detection, real-time analytics, and S3 archival. Design the Kinesis architecture.

Writes are ~24 MB/s, so use Kinesis Data Streams On-Demand or sufficient provisioned shards with headroom. Use a well-distributed partition key such as session ID or user ID to maintain per-user ordering. Use KPL aggregation for many small events. Use Enhanced Fan-Out for consumers that need dedicated low-latency reads, and Firehose as the S3 delivery consumer, converting events to Parquet and using dynamic partitioning. Retain records in Kinesis for replay and monitor `IteratorAge` and throttling metrics.

Q2. Consumers are falling behind and `IteratorAgeMilliseconds` is continuously increasing, but producers are writing normally. What are the possible causes and fixes?

First check whether the consumer is processing records too slowly. For Lambda, increase the batch size or parallelization factor, reduce per-record processing, and check for retries or a poison record; enable bisect on error where appropriate. For KCL, add workers so processing capacity keeps up with the shard count. If the bottleneck is shared read throughput, consider Enhanced Fan-Out. If the consumer is waiting on a slow downstream system, batch or buffer the writes and improve the downstream processing.

Q3. One Kinesis shard is constantly throttling while the other shards are mostly idle. What happened and how would you fix it?

This is most likely a hot partition key. A low-cardinality key or a single high-volume customer may be sending too much data to one shard. First examine the partition-key distribution. Choose a better high-cardinality key, or use key salting if appropriate. If ordering by entity is required, carefully design the key so ordering is preserved. You can also split the hot shard where appropriate, but simply adding capacity does not fix a poorly distributed key. On-Demand mode does not eliminate the hot-key problem.

Q4. A team says they need to reprocess Kinesis events from yesterday because a bug was discovered in the consumer. How would you handle it?

If the records are still within the stream's retention period, start a consumer from the required point using `AT_TIMESTAMP` or an appropriate sequence number and run the corrected processing logic. Make the downstream process idempotent so replay does not create duplicates. For long-term recovery, persist the raw events to S3, typically using Firehose or another consumer, because Kinesis retention is a replay window rather than permanent storage.

Q5. You need to send the same Kinesis events to fraud detection, S3, and a real-time dashboard. Would you use one consumer or multiple consumers?

Use multiple independent consumers so each application can process the same stream independently. Use Lambda or Flink for fraud detection, Firehose for managed delivery to S3, and Lambda/KCL/Flink for the dashboard depending on processing requirements. If several consumers require low-latency reads, use Enhanced Fan-Out so they do not compete for shared shard read throughput. This decouples the consumers so one slow or failed application does not stop the others.

MENTAL MAP | KINESIS = SHARDS × (ORDER + THROUGHPUT); FIREHOSE = LAND IT

 KDS	• Shard limits: 1 MB/s write or 1,000 records/s, 2 MB/s read (shared across consumers) • Record size: up to 1 MB (10 MiB with large-record support) • Partition key → shard (hash); ordering guaranteed within a shard • Modes: provisioned (manual shards) on-demand (auto-scale) • Retention: 24 hours (default) → 7 days → 365 days (extra cost) • Replay: start from LATEST TRIM_HORIZON AT_TIMESTAMP sequence number
 PRODUCERS	• PutRecord / PutRecords (up to 500 records or 5 MB per batch) • Kinesis Producer Library (KPL): aggregation, retries, metrics • Kinesis Agent: tails log files • Native sources: CloudWatch Logs, EventBridge, IoT Core, DMS, applications (SDKs) • Handle ProvisionedThroughputExceededException with backoff and retries • Design a high-cardinality partition key (e.g., user id); avoid hot keys
 CONSUMERS	• Lambda event source mapping: batches, parallelization (up to 10 per shard), retries, on-failure • KCL (DynamoDB leases): custom consumers, multi-language, checkpointing • Enhanced fan-out (EFO): dedicated 2 MB/s per shard per consumer, low latency • Glue Streaming / Flink: real-time ETL and stateful processing • Firehose: reads the stream, buffers, delivers to S3/Redshift/OpenSearch/etc.
 SCALE	• UpdateShardCount: double/half shards (scales uniform) • SplitShard / MergeShard: target specific hot or cold shards • Monitor: IncomingBytes, IncomingRecords, WriteProvisionedThroughputExceeded, ReadProvisionedThroughputExceeded, IteratorAgeMilliseconds (consumer lag) • On-demand removes resharding effort but does not fix hot keys
 FIREHOSE	• Buffer: size (1–128 MB for S3) or interval (60–900 s); delivers when either threshold is met • Transform: Lambda per batch (max 6 MB response, 5 min timeout); built-in JSON → Parquet/ORC via Glue • Dynamic partitioning: create S3 prefixes (e.g., year=/month=/day=/) from record fields • Destinations: S3, Redshift (via S3 + COPY), OpenSearch, Iceberg tables, Splunk, Datadog, HTTP • No replay of successful deliveries; failed records go to an S3 error prefix • No ordering guarantees; primarily for managed delivery, not custom stream processing

YOUR NOTES

7 AWS Lambda

Small, stateless, event-driven compute. React to events; do not run the warehouse in it.

7.1 What it does and where it fits

AWS Lambda is a serverless compute service that runs code in response to events and automatically scales as events arrive. In data engineering, it is commonly used to react when an S3 file lands, process a Firehose batch, consume Kinesis records, call an API and write results to S3, trigger a Glue job or Step Functions workflow, or validate files. Lambda is best for small, short-lived, event-driven processing, not large-scale workloads such as a 2 TB join; for those, use distributed processing services such as Glue/Spark.

ANALOGY

Lambda is like a fleet of contractors waiting on call. Each contractor shows up only when a doorbell rings (trigger), works inside a small room called the execution environment, and leaves when the job is finished. Each job can run for a maximum of 15 minutes. If 1,000 doorbells ring at once, Lambda can create many contractors to handle the work in parallel - this is concurrency. Provisioned concurrency is like keeping some contractors already inside the room and ready to work, which helps reduce startup delay.

7.2 Key components

Function, runtime, execution environment

- **Handler:** The handler is the entry point of a Lambda function and receives the event (input data) and context (information about the invocation and runtime). Code outside the handler runs during environment initialization and can be reused when the same execution environment handles another invocation, so it is a good place to create SDK clients, database connections, and load configuration.
- **Runtimes:** Lambda provides managed runtimes such as Python, Node.js, Java, .NET, and Ruby, and you can also use custom/OS-only runtimes or deploy a container image up to 10 GB. Lambda Layers allow you to package and share common dependencies such as pandas and pyarrow across functions.
- **Cold Start:** A cold start occurs when Lambda creates a new execution environment and initializes the function before processing an invocation. Cold starts can be reduced by keeping packages small, using lazy imports, minimizing initialization work, and using features such as Provisioned Concurrency or supported SnapStart configurations when appropriate.
- **/tmp Storage:** Lambda provides ephemeral /tmp storage for temporary files, with 512 MB by default and configurable up to 10,240 MB (10 GB). Data in /tmp should not be treated as permanent storage;

use Amazon EFS when you need shared file storage or storage beyond the local temporary environment.

- **Versions & Aliases:** Versions create immutable versions of a Lambda function, while aliases such as dev, staging, and prod point to specific versions. This makes deployments safer because you can test a new version, shift traffic, and quickly roll back if something goes wrong.
- **Environment Variables & Secrets:** Environment variables store configuration such as database names, S3 paths, or feature flags, with a 4 KB total limit. Sensitive values should generally be stored in AWS Secrets Manager or Systems Manager Parameter Store rather than hard-coded in the function, and frequently used secrets/configuration can be cached to reduce repeated service calls.

Limits to have in your head

Limit	Value	Consequence
Timeout	900 seconds (15 minutes) maximum	Long-running ETL should use Glue, EMR, Batch, or be broken into smaller steps with Step Functions.
Memory	128 MB – 10,240 MB (10 GB)	CPU allocation increases with memory. More memory can sometimes be more cost-effective because the function finishes faster.
Payload	6 MB for synchronous request/response; 1 MB for asynchronous invocation	Pass S3 keys/references rather than large datasets directly through Lambda events.
Concurrency	1,000 concurrent executions per Region by default (quota can be increased)	A sudden spike or noisy function can consume available concurrency. Use reserved concurrency to protect critical functions.
Deployment package	50 MB zipped / 250 MB unzipped for ZIP deployment; 10 GB for container images	Use container images for functions with very large dependencies.

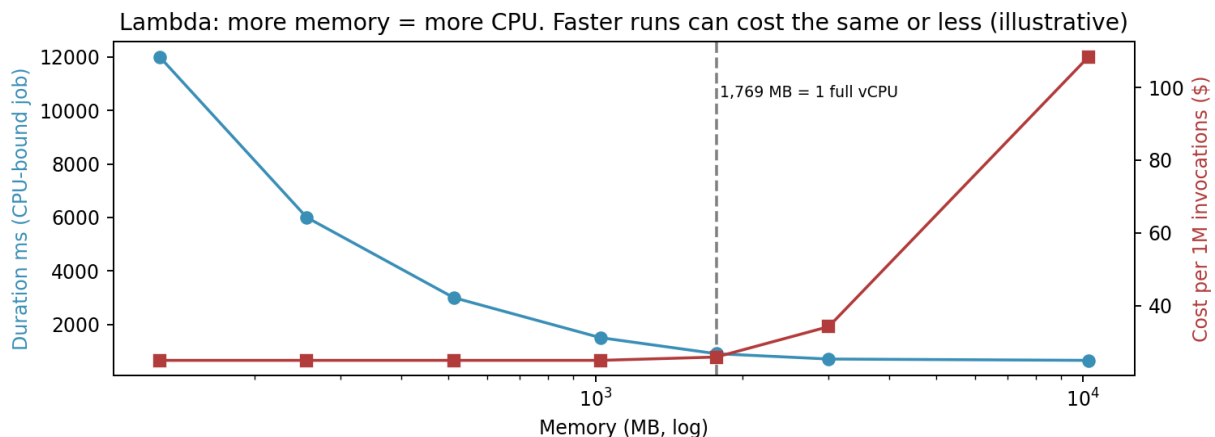


Figure 3. Illustrative CPU-bound function: raising memory shortens duration, so cost per invocation can stay flat or fall. Use AWS Lambda Power Tuning to find the knee.

Event sources and invocation models

Model	Sources	Behaviour on failure
Model	Sources	Behaviour on failure
Synchronous (request/response)	API Gateway, Application Load Balancer (ALB), direct Invoke, Cognito	The caller receives the result/error and is responsible for retrying if required.
Asynchronous (event-driven)	S3 events, SNS, EventBridge, and other supported event sources	Lambda queues the event and retries failed invocations up to 2 additional times by default. Events can be retained for up to 6 hours. Configure an on-failure destination or DLQ (SQS/SNS). The function should be idempotent because an event can be processed more than once.

S3 Triggers & Event Routing

- **S3 → Lambda:** Simple option for triggering processing when a file arrives.
- **S3 → EventBridge:** Better when you need **filtering and multiple targets**.
- **S3 → SQS → Lambda:** Use when you need **buffering and controlled concurrency** during high-volume file arrivals.

Concurrency

- **Concurrency = events/second × average execution time.**
- **Reserved concurrency:** Limits how many instances a function can use and protects downstream systems.
- **Provisioned concurrency:** Keeps environments warm to reduce **cold-start latency**.
- **Kinesis:** Concurrency depends on **shards × parallelization factor**.
- **SQS:** Control maximum concurrency to avoid overwhelming databases or APIs.

Error Handling & Retries

- Design Lambda processing to be **idempotent** because events can be delivered more than once.
- **Transient errors** → retry.
- **Permanent errors** → send to a **DLQ/failure destination** for investigation.
- **Kinesis:** Use **BisectBatchOnFunctionError**, maximum retry attempts, and maximum record age to prevent bad records from blocking processing.
- **SQS:** Failed messages become visible again after the visibility timeout and can move to a **DLQ** after `maxReceiveCount`.
- Use **Step Functions** for multi-step workflows with retries, branching, and parallel processing.

Monitoring

- Monitor **Errors, Throttles, Duration, and ConcurrentExecutions**.
- For Kinesis consumers, monitor **IteratorAgeMilliseconds** to detect consumer lag.
- Use **CloudWatch Logs** for troubleshooting.

Execution Role (IAM)

- **Execution role:** Defines what the Lambda function is allowed to do, such as reading S3 or starting a Glue job.
- **Resource-based policy:** Defines who is allowed to **invoke the Lambda function**.
- Follow **least privilege** and give the function only the permissions it actually needs.

REMEMBER: Concurrency = protect downstream systems · Idempotency = safe retries · DLQ = failed events · IteratorAge = Kinesis lag · Execution role = what Lambda can do.

Writing a Lambda that lists an S3 prefix, downloads every file, and processes them in a loop. It works in dev and times out in prod. The senior pattern: one event per object, or Step Functions Distributed Map, or hand it to Glue once volume exceeds what a single environment can do in a few minutes.

7.3 Scenario questions

Q1. An S3 bucket receives 10,000 files at once and triggers Lambda. The database starts getting overwhelmed. What would you do?

→ Put SQS between S3 and Lambda to buffer the workload and control Lambda concurrency. Set a suitable maximum concurrency on the SQS event source mapping so Lambda does not create too many database connections.

Q2. A Lambda function is taking 12 minutes to process a large file and sometimes exceeds the 15-minute timeout. How would you redesign it?

→ Lambda is not suitable for this workload. Break the file into smaller units or move the processing to AWS Glue/Spark or another distributed processing service. Lambda should handle lightweight, event-driven processing rather than large ETL jobs.

Q3. A Lambda function processes Kinesis records, but IteratorAgeMilliseconds keeps increasing. What would you investigate?

→ The consumer is falling behind. Check Lambda duration, errors, throttling, batch size, and parallelization factor. Increase processing capacity if needed, and consider Enhanced Fan-Out if shared Kinesis read throughput, is the bottleneck.

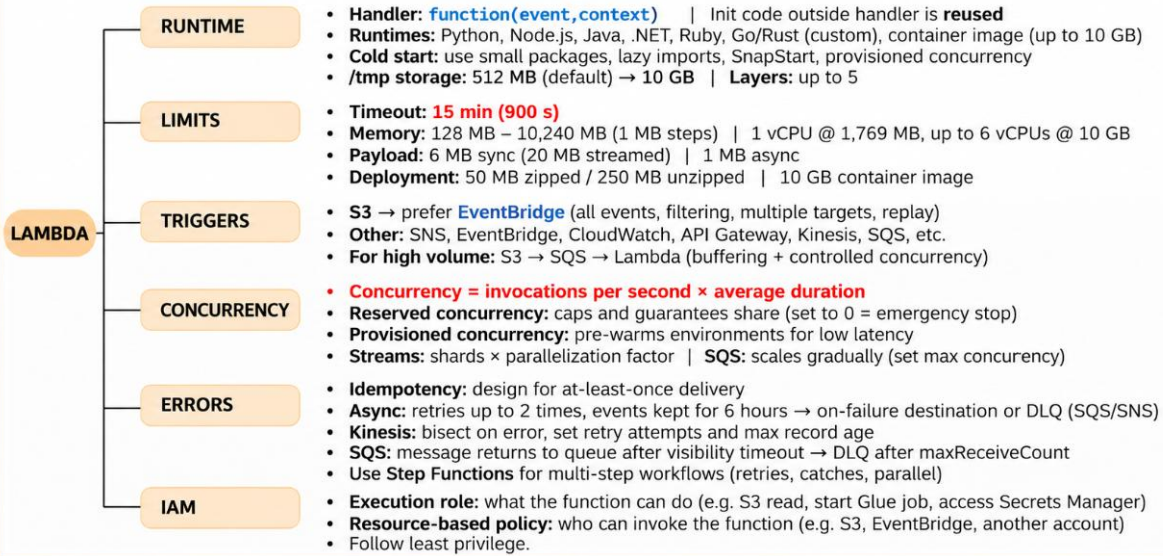
Q4. A Lambda function sometimes processes the same event twice and creates duplicate records in the database. How would you fix it?

→ Design the function to be idempotent. Use a unique event ID or business key to detect duplicates and make the database operation safely repeatable. This is important because event-driven architectures generally provide at-least-once delivery.

Q5. A Lambda API is slow for the first request but fast for subsequent requests. What is likely happening and how would you improve it?

→ This is likely a cold start. Reduce package size and initialization work, use lazy imports where appropriate, and consider Provisioned Concurrency for latency-sensitive workloads.

MENTAL MAP | LAMBDA = EVENT → SMALL STATELESS WORK → HAND OFF



YOUR NOTES

8 Amazon CloudWatch

Metrics, logs, alarms. Alarm on symptoms the business feels, not just on CPU.

8.1 What it does and where it fits

AWS CloudWatch is AWS's monitoring and observability service that collects metrics, logs, and events from AWS services and applications, allowing you to monitor pipeline health, failures, performance, capacity, freshness, lag, and cost-related metrics through dashboards, queries, graphs, and alarms. In a data engineering pipeline, CloudWatch helps you detect issues such as a slow Glue job, Kinesis consumer lag, Lambda errors, or excessive resource usage; it works alongside EventBridge for event routing, CloudTrail for AWS API auditing, X-Ray for distributed tracing, and SNS for sending notifications.

ANALOGY

CloudWatch is like a hospital monitoring station. Metrics are the vital signs recorded over time, logs are the nurses' notes, alarms alert the team when a vital sign crosses a threshold, and dashboards show the overall health on monitoring screens. EventBridge is like the hospital intercom, routing an event such as "patient in Room 4 needs help" to the appropriate team.

8.2 Key components

Metrics

- **Metric**
 - Identified by namespace, metric name, and dimensions.
 - Examples: AWS/Glue, AWS/Kinesis, with dimensions such as JobName or StreamName.
 - Metrics are collected as data points over a specific period.
 - Common statistics include Sum, Average, and percentiles such as p99.
 - Standard resolution is 1 minute; high-resolution metrics can be 1 second.
 - Metrics are retained for up to 15 months, with older data stored at coarser resolutions.
- **Custom Metrics**
 - Create custom metrics using PutMetricData.
- **Embedded Metric Format (EMF)** allows you to write metric data as JSON in CloudWatch Logs, which CloudWatch extracts into metrics.
 - Useful pipeline metrics include:
 - rows_processed
 - rows_rejected
 - freshness_seconds
 - run_duration

- **Metric Math**

- Combines metrics to calculate useful values such as rates, ratios, and percentages.
- Example: instead of alarming on the number of errors, alarm when errors / invocations > 2%.
- Supports functions such as FILL and anomaly-detection bands.

- **Cost**

- Custom metrics are charged based on the number of metrics.
- Keep dimension cardinality low.
- Avoid dimensions such as FileName that could create thousands or millions of unique metric combinations.

Service	Metrics that matter for a data pipeline	Alarm idea
Service	Metrics that matter for a data pipeline	Alarm idea
Glue	Failed tasks, JVM heap usage, bytes read, job run state, DPU seconds	Job FAILED event; heap > 90% sustained; run duration > 2× baseline
Kinesis	GetRecords.IteratorAgeMilliseconds, WriteProvisionedThroughputExceeded, IncomingBytes, PutRecords.FailedRecords	IteratorAge > 60 sec for 5 min; any throughput exceeded
Firehose	DeliveryToS3.Success, DeliveryToS3.DataFreshness, ThrottledRecords	DataFreshness > 3× buffer interval; delivery success < 1
Lambda	Errors, Throttles, Duration (p95), ConcurrentExecutions, IteratorAge, DeadLetterErrors	Errors/Invocations > 1%; throttles > 0; DLQ delivery failures
Redshift	CPUUtilization, PercentageDiskSpaceUsed, QueueLength, WLMQueueWaitTime, ConcurrencyScalingSeconds; Serverless: ComputeCapacity, QueriesQueued	Disk > 80%; queue wait p90 > 30 sec
Athena	DataScannedInBytes, TotalExecutionTime, QueryQueueTime per workgroup	Daily bytes scanned exceed budget

Logs and Logs Insights

Log Group & Log Streams

- A log group contains multiple log streams, typically separated by application, function, instance, or execution environment.
- Set a retention period for every log group. The default is never expire, which can increase costs.
- CloudWatch Logs has Standard and Infrequent Access log classes, with Infrequent Access designed for lower-cost storage with fewer features.

AWS Service Logs

- Glue writes driver and executor logs to CloudWatch, including output/error logs and continuous logging when enabled.
- Lambda writes logs to `/aws/lambda/<function-name>`.
- Use structured JSON logs so CloudWatch Logs Insights can easily filter and analyze individual fields.

CloudWatch Logs Insights

- Used to search, filter, aggregate, and analyze logs interactively.
- Common commands include `fields`, `filter`, `parse`, `stats`, `sort`, and `limit`.
- Example: count errors by job and hour.
- Logs Insights charges based on data scanned, so limit the time range and filter data early.

Metric Filters

- Convert matching log patterns into CloudWatch metrics.
- Example: count occurrences of `OutOfMemoryError` and create an alarm when the count becomes too high.

Subscription Filters

- Stream logs in near real time to services such as Kinesis, Firehose, Lambda, or OpenSearch for further processing or centralized analysis.
- Logs can also be exported to S3 for cheaper long-term storage and querying with Athena.

Sensitive Data

- Avoid logging PII, passwords, tokens, or complete data records.
- Use CloudWatch data protection policies where appropriate to detect and mask sensitive information.

Alarms

Alarm States

A CloudWatch alarm monitors a metric or metric-math expression over a defined number of evaluation periods and compares it against a threshold. It can be OK, ALARM, or INSUFFICIENT_DATA. You can also control how missing data is treated—for example, use `notBreaching` for sparse job metrics and `breaching` for heartbeat or freshness checks where missing data may indicate a problem.

Composite Alarms

Composite alarms combine multiple alarms using AND/OR logic to reduce unnecessary alerts. For example, instead of alerting whenever Kinesis lag increases, you could alert only when lag is high AND errors are increasing, making the alert more meaningful.

Anomaly Detection

Anomaly detection learns the normal historical pattern of a metric and creates an expected range around it. This is useful for metrics with predictable patterns, such as daily row counts or pipeline duration, where a fixed threshold may generate too many false alarms.

Alarm Actions

When an alarm enters the ALARM state, it can trigger actions such as sending notifications through SNS, invoking Lambda for automated remediation, or creating an operational issue through Systems Manager. These actions can also be integrated with incident-management and notification systems.

Data Engineering Design Rule

For data pipelines, focus alarms on freshness, completeness, lag, and failure, because these represent the actual business impact. CPU or memory problems are often technical causes, while a stale or missing Gold table is the problem the business ultimately notices.

Dashboards and EventBridge

CloudWatch Dashboards provide a central view of pipeline health using graphs, numbers, log queries, and alarm status, and can bring together information across regions and accounts. For a data pipeline, a dashboard should focus on what an on-call engineer needs to know quickly, such as ingestion rates, consumer lag, job duration versus baseline, failure counts, and data freshness for Gold tables. EventBridge connects pipeline events to actions - for example, Glue job state changes, Step Functions status changes, S3 object creation, Redshift events, or scheduled rules can trigger SNS, Lambda, Step Functions, or SQS, with filtering and event archiving/replay available.

Monitoring a data pipeline end to end

Layer	What to measure	How
Ingestion	Records/bytes received, producer errors, consumer lag	Kinesis/Firehose metrics; alarm on IteratorAge and throughput exceeded
Landing	Files received vs expected, file-size distribution	S3 events counted through EMF; anomaly alarm on daily object count
Transformation	Job success/failure, duration vs baseline, rows in/out, rejected rows, DPU usage	Glue job metrics + EventBridge job state changes + custom EMF metrics
Data Quality	Null rates, uniqueness, referential integrity, schema drift	Glue Data Quality (DQDL) or Deequ/Great Expectations; fail the pipeline on critical rules
Serving	Table freshness, query latency, queue waits, bytes scanned	Scheduled Lambda/Athena freshness check emitting metrics; Redshift/Athena metrics
Cost	DPU hours, bytes scanned, RPU usage, shard hours	CloudWatch metrics + AWS Cost Explorer/Budgets alarms

A dashboard with 40 widgets and no alarms. Or alarms that fire so often the channel is muted. Say "we tuned alarms until every page was actionable" and be ready with an example of an alarm you deleted.

8.3 Scenario questions

Q1. Your daily Gold table is usually ready by 7 AM, but today it is still not updated at 8 AM. What would you monitor and how would you troubleshoot it?

→ I would monitor data freshness as a custom CloudWatch metric and create an alarm when freshness exceeds the expected threshold. Then I would trace the pipeline backwards: check the Glue job status and duration, S3 file arrival/completeness, Kinesis lag if streaming is involved, and any downstream Redshift/Athena issues. The key is to alarm on the stale data, not just CPU or infrastructure metrics.

Q2. A Glue job normally takes 20 minutes but suddenly takes 60 minutes. How would you detect and investigate this?

→ Monitor Glue job duration and compare it against the normal baseline using CloudWatch. I would then check job state, failed tasks, bytes read, memory/heap usage and DPU usage. I would also investigate changes in input volume, small files, data skew, or resource constraints rather than simply increasing the number of workers.

Q3. Your Kinesis pipeline is processing data successfully, but the data in the S3 Gold table is becoming increasingly delayed. What would you check?

→ First check Kinesis IteratorAgeMilliseconds to determine whether the consumer is falling behind. Then check Lambda/Glue processing duration, errors and throttling, followed by Firehose DataFreshness and delivery metrics. If Kinesis lag is increasing, fix the consumer capacity or processing bottleneck before looking at the S3 layer.

Q4. Your pipeline receives 1,000 files every day, but today only 700 arrived. How would you detect this automatically?

→ Track expected versus actual file counts using S3 events and publish the count as a CloudWatch custom metric, for example through EMF. Use an anomaly-detection alarm or an expected-count threshold to identify the unusual drop. I would also check whether the upstream ingestion process failed rather than assuming the missing files are a downstream problem.

Q5. Your AWS pipeline suddenly becomes much more expensive. How would you use CloudWatch to investigate?

→ I would look at the main cost drivers: Glue DPU usage, Athena bytes scanned, Redshift compute/RPU usage, and Kinesis shard usage. CloudWatch metrics can show where resource consumption increased, while Cost Explorer and AWS Budgets help identify the actual cost increase. I would then correlate the increase with changes in data volume, query patterns, job duration, or infrastructure configuration.

REMEMBER: Monitor the pipeline from Ingestion → Landing → Transformation → Data Quality → Serving → Cost. A senior engineer monitors the business outcome (freshness, completeness, quality) as well as the technical causes.

MENTAL MAP | CLOUDWATCH = METRICS + LOGS + ALARMS + EVENTS (EVENTBRIDGE ROUTES)

COMPONENTS	Metrics	Numerical data points over time (e.g., CPU, job duration, rows processed).
	Logs	Text logs from services, applications and custom code.
	Alarms	Monitor metrics or log-based metrics and trigger actions when thresholds are breached.
	EventBridge	Routes events (e.g., Glue job state change, S3 object created) to the right target (SNS, Lambda, Step Functions, etc.).
USE CASES	Monitoring	Track pipeline health, performance and data freshness.
	Alerting	Notify on failures, data delays, or unusual patterns.
	Automation	Trigger workflows (e.g., retry job, notify team, create ticket) using EventBridge.
KEY SERVICES INTEGRATIONS	AWS Services	Glue, Kinesis, Firehose, Lambda, Redshift, Athena, S3, Step Functions, ECS/EKS, RDS, etc.
	External Tools	SNS (email/Slack via Chatbot), PagerDuty, Jira, ServiceNow, Datadog (via integrations).
KEY METRICS (EXAMPLES)	Glue	JobRunState, DPU seconds, FailedTasks, JobDuration.
	Kinesis	IteratorAgeMilliseconds, IncomingBytes, WriteProvisionedThroughputExceeded.
	Lambda	Errors, Throttles, Duration (p95), ConcurrentExecutions.
	Redshift	CPUUtilization, PercentageDiskSpaceUsed, WLMQueueWaitTime.
	S3	NumberOfObjects, BucketSizeBytes, ReplicationLatency.

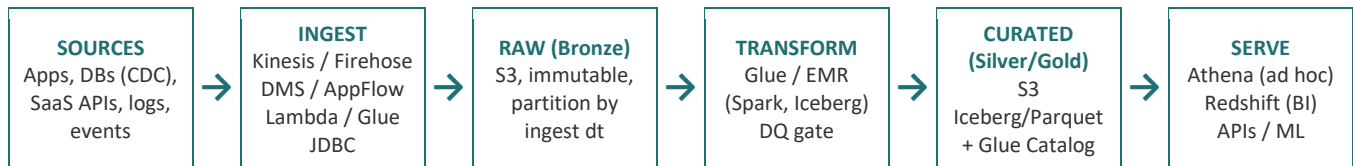
GOAL DETECT ISSUES EARLY, REDUCE MTTR, AND ENSURE RELIABLE, COST-EFFICIENT DATA PIPELINES.

YOUR NOTES

9 End-to-End Data Architecture

Putting it together: the design questions a senior interview actually asks

9.1 The reference architecture



Cross-cutting: **orchestration** (Step Functions / MWAA / Glue Workflows), **governance** (Lake Formation, Glue Catalog, tags), **security** (IAM roles, KMS, VPC endpoints), **observability** (CloudWatch, EventBridge), **IaC** (CDK/Terraform), **CI/CD** (OIDC roles, tests on sample data).

ANALOGY: Think of a restaurant kitchen. Deliveries arrive through the back door and go straight into cold storage untouched (Raw/Bronze). The prep cooks clean, validate, and organize the ingredients into ready-to-use components (Silver), and the chefs turn them into finished dishes (Gold). Customers and the dining room (Athena/Redshift) consume the finished dishes without needing to enter the kitchen. IAM and Lake Formation act like the access controls that decide who can enter each area and what they can access, while CloudWatch is the manager's dashboard, showing things that matter such as ticket times, delays, failures, and workload-not every detail happening inside the kitchen.

9.2 The core decisions

Batch vs streaming

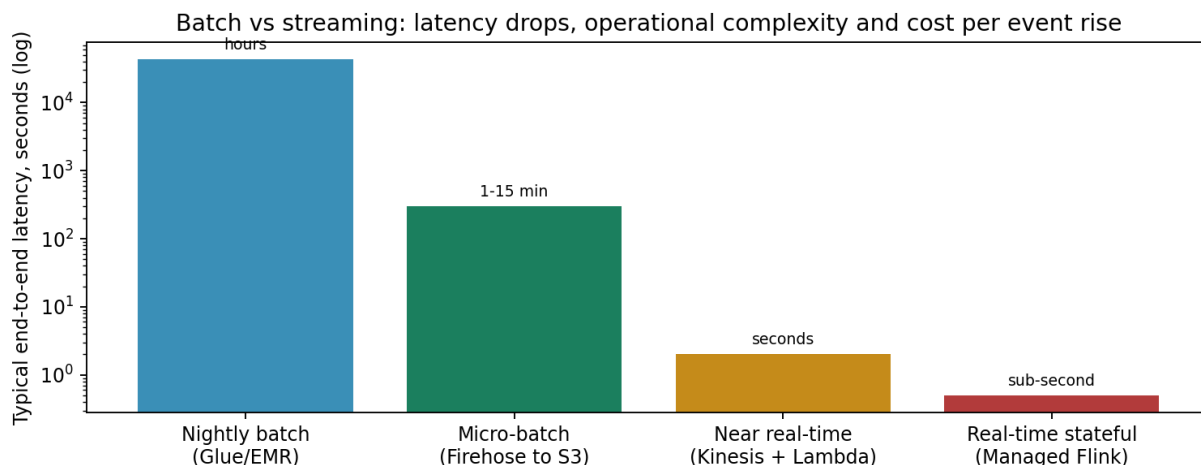


Figure 4. Latency vs complexity. Choose the slowest option that meets the business SLA.

Question to ask	Batch (Glue / EMR / Redshift loads)	Streaming (Kinesis / Flink / Lambda / Glue Streaming)
What latency does the consumer need?	Minutes to hours are acceptable, such as daily reports or scheduled analytics.	Seconds to minutes are required, such as fraud detection, operational dashboards, or real-time personalization.
How is the source shaped?	Files, snapshots, database extracts, bulk exports.	Events, CDC records, clickstream, IoT/device events.
How is reprocessing handled?	Usually simple: rerun the affected partition or batch.	Replay events from stream retention; state, ordering, and time windows can make reprocessing more complex.
Cost profile	Compute runs when needed, so it is often more cost-effective for periodic workloads.	Streaming infrastructure may run continuously; can be cost-effective for steady, high-volume real-time workloads.
Correctness	Easier to achieve deterministic processing using partition-level writes/overwrites and idempotent jobs. Exactly-once is not automatic.	At-least-once processing is common; use idempotency, checkpoints, and watermarks where required. Some streaming technologies can provide stronger processing guarantees with the right design.
Typical answer	Default choice for most scheduled analytics and ETL when real-time SLA is not required.	Use when the business SLA genuinely requires real-time or near-real-time processing. Often, micro-batch or frequent Firehose delivery is sufficient instead of a complex streaming architecture.

The senior move: propose **Lambda-style hybrid without the duplication**: stream raw events to S3 via Firehose (micro-batch), build Silver/Gold in batch from the same raw zone, and add true streaming consumers only for the two or three use cases that need seconds. Same raw data, one truth.

ETL vs ELT

Aspect	ETL (Transform before Load)	ELT (Load first, Transform later)
Where transforms run	Glue, EMR, Spark before loading into the target	Redshift SQL, Athena CTAS, dbt after raw data is loaded
Strength	Good for large, messy, semi-structured data, complex parsing, cleansing, and transformations before serving	Faster to develop, SQL-friendly, and lets the target platform handle transformations

Aspect	ETL (Transform before Load)	ELT (Load first, Transform later)
Weakness	More engineering code and Spark infrastructure/skills; transformations can take longer to develop	Can increase warehouse/query compute costs and requires the target platform to handle the workload
AWS example	S3 → Glue/Spark → S3 Iceberg → Redshift/Athena	S3/raw data → Redshift → SQL/dbt models
Best suited for	Heavy processing such as parsing, cleansing, deduplication, schema normalization, and PII handling	Business transformations, joins, aggregations, and analytics models

Incremental processing across the stack

Stage	Mechanism	Idempotency Guard
Source → Raw	CDC using AWS DMS or event streams; API ingestion using a high-water mark such as timestamp or ID	Keep Raw append-only where possible; use event IDs/source keys for deduplication
Stage	Mechanism	Idempotency Guard
Source → Raw	CDC using AWS DMS or event streams; API ingestion using a high-water mark such as timestamp or ID	Keep Raw append-only where possible; use event IDs/source keys for deduplication
Raw → Silver	Glue Job Bookmarks, partition-based processing, or Iceberg incremental reads	Use MERGE on business key + appropriate change/event timestamp, or safely overwrite affected partitions
Silver → Gold	Recompute only affected partitions, including a window for late-arriving data (e.g., last 3 days)	Partition overwrite of the recomputed window rather than repeatedly appending the same results

Data quality

Gate, don't just report: Critical data-quality checks should block the pipeline from promoting data to Silver when they fail. Typical checks include schema validity, primary/business-key uniqueness, row counts within an expected range, mandatory columns being non-null, and referential integrity. Warning-level checks can record metrics and alerts without stopping the pipeline.

Data Quality Tools

On AWS, Glue Data Quality supports rule-based checks using DQDL (Data Quality Definition Language) and can run against Glue Data Catalog tables or within Glue jobs. Results can be integrated with CloudWatch

and EventBridge. Other commonly used tools include Deequ, Great Expectations, and dbt tests for ELT workflows.

Schema Evolution

Iceberg supports controlled schema evolution, including operations such as adding and renaming columns. For streaming data, Glue Schema Registry can help manage and validate schemas. Unexpected schema changes should be detected and handled according to the data contract rather than silently accepted.

Quarantine Pattern

When individual records fail quality checks, send them to a quarantine/rejected table or location along with the failure reason. Track the number of rejected records as a metric, and never silently discard bad data.

Lineage and Ownership

Maintain metadata such as data owner, SLA, sensitivity/PII classification, and data quality status in the catalog or governance layer. Define a freshness SLO for important Gold tables so teams know how current the data is expected to be.

Orchestration and dependencies

Option	Fit	Trade-off
Glue Workflows + Triggers	Simple Glue-centric workflows involving jobs, crawlers, and triggers	Less flexible for complex branching and workflows involving many AWS services
Step Functions	Serverless orchestration across Glue, Lambda, Athena, Redshift, EMR, and other AWS services; supports retries, error handling, branching, parallel execution, and Distributed Map for large-scale fan-out	Workflow definitions can become complex for very large workflows; less suited than Airflow for sophisticated DAG management and backfills
Amazon MWAA (Managed Airflow)	Complex DAGs, dependencies, backfills, sensors, scheduling, and workflows involving many teams and tools such as dbt	More operational overhead and ongoing cost because the Airflow environment must be managed and maintained
EventBridge Scheduler / EventBridge Events	Time-based and event-based triggers that start Lambda, Step Functions, Glue and other supported targets	EventBridge is primarily a trigger/routing service, not a complete workflow orchestration engine

Dependency rules worth stating: dependencies on **data readiness**, not on clock time (wait for the partition or the success event, not "run at 6 am and hope"); every task idempotent and retry-safe; SLAs on Gold tables with alerting upstream of the consumer; backfill is the same DAG with a date parameter.

Security, performance and cost together

Pillar	Non-negotiables	Senior-level extras
Security	One IAM role per workload; KMS encryption; S3 Block Public Access; TLS enforced; secrets in Secrets Manager; VPC endpoints for private access where required; CloudTrail enabled	Lake Formation column/row-level grants and LF-Tags; PII tokenisation in Silver; cross-account access using resource policies and <code>aws:PrincipalOrgID</code> where appropriate; data-perimeter SCPs
Performance	Parquet/Iceberg; target 128 MB–1 GB file sizes; partition on common filter columns; co-locate Redshift joins where appropriate; broadcast small dimension tables	Compaction schedules; partition projection; materialized views for dashboards; Athena Capacity Reservations when predictable query capacity is required; right-sized Glue workers with Auto Scaling
Cost	S3 Lifecycle rules; log retention; Athena bytes-scanned limits; use Glue Flex for suitable non-urgent workloads; Redshift Serverless for variable workloads; Kinesis On-Demand for variable workloads	Cost-allocation tags per pipeline; AWS Budgets alarms; KPL aggregation for small Kinesis records; S3 Bucket Keys; clean up incomplete multipart uploads; Spot Instances on suitable EMR workloads; measure cost per table/pipeline per day

9.3 Scenario questions (whiteboard style)

Q1. Design a data platform for a telecom company that receives call events and billing CDC, and needs both daily reporting and a near-real-time operations dashboard.

Answer:

I would use Kinesis Data Streams for real-time call events and use a well-distributed partition key such as `call_id`. Firehose can deliver the events to S3 in Parquet for the raw layer, while a Lambda or Flink consumer handles the near-real-time dashboard metrics. For billing CDC, I would use AWS DMS → S3 Raw. Glue would transform Raw → Silver → Gold, with deduplication, PII handling, and data-quality checks. Gold data can be queried through Athena, while Redshift can serve BI workloads. Step Functions can orchestrate the batch workflow, and CloudWatch can monitor freshness, failures, and streaming lag.

Q2. The platform's AWS cost has increased by 30%. How would you investigate and reduce the cost without affecting SLAs?

Answer:

I would first identify the largest cost drivers using AWS Cost Explorer and cost-allocation tags, then investigate areas such as Athena bytes scanned, Glue DPU usage, Redshift compute, Kinesis capacity, S3 storage, and CloudWatch Logs. I would look for unnecessary full scans, poor partitioning, small files, oversized Glue jobs, idle Redshift capacity, excessive Kinesis capacity, and logs with unlimited retention. Then I would apply targeted optimizations such as Parquet/partitioning, Glue right-sizing or Flex for suitable jobs, Redshift Serverless for variable workloads, Kinesis On-Demand for variable traffic, S3 Lifecycle rules, and log retention.

Q3. A daily Gold table receives data late because some source records arrive one or two days after their event date. How would you handle this?

Answer:

I would partition Silver using the event date rather than arrival date and design the Gold job to recompute a small lookback window, such as the last 3 days. The affected Gold partitions would be safely overwritten rather than appended again. The lookback period should be based on actual arrival-lag statistics, which I would monitor as a data-quality/freshness metric. This allows late-arriving records to be incorporated without rebuilding the entire historical dataset.

Q4. A team wants to build the entire platform using streaming because it is "more future-proof." Would you agree?

Answer:

No, not automatically. I would first ask which business requirements actually need seconds or sub-minute latency. Streaming adds infrastructure cost and complexity around ordering, state, replay, error handling, and monitoring. I would use streaming where the SLA requires it, but use batch or micro-batch for workloads where minutes or hours are acceptable. For example, Firehose delivering data to S3 frequently may be sufficient instead of building a full streaming-processing layer.

Senior thinking: Choose the architecture based on the business SLA, not because the technology is newer.

Q5. How would you make a data platform safe to change and deploy?

Answer:

I would manage infrastructure using Terraform or CDK, store code in Git, and use CI/CD with automated testing. Transformation logic should have unit tests, and integration tests should run against representative sample data. Data-quality checks should act as deployment or pipeline gates for critical rules. For production changes, use versioning and controlled deployment strategies so new code can be tested and rolled back safely. Every important CloudWatch alarm should also have a runbook explaining what the engineer should check and how to recover.

REMEMBER: Design → Security → Data Quality → Performance → Cost → Monitoring → Recovery.

YOUR NOTES

Appendix: Night-Before Revision

A.1 Numbers worth knowing cold

Item	Value
S3 object size	5 GB single PUT, 5 TB with multipart; 3,500 writes / 5,500 reads per second per prefix
S3 minimum durations	IA: 30 days; Glacier Instant/Flexible: 90 days; Deep Archive: 180 days; Intelligent-Tiering ignores objects < 128 KB
Target file size	128 MB to 1 GB Parquet, Snappy or ZSTD
Glue	1 DPU = 4 vCPU + 16 GB; Glue 6.0 = Spark 4.1, Python 3.13, 30% cheaper; Flex ~35% cheaper still
Athena	~\$5 per TB scanned, 10 MB minimum; CTAS max 100 partitions per statement; Capacity Reservations: 1-min billing, 4 DPU minimum
Redshift	Distribution styles: KEY / ALL / EVEN / AUTO; sort keys: compound / interleaved / AUTO; RA3 and Serverless use managed storage; Python UDFs unsupported after June 2026
Kinesis shard	1 MB/s or 1,000 records/s in; 2 MB/s out; retention 24 h default, 7 d extended, 365 d long-term; on-demand default cap 200 MB/s (1 GB/s on request)
Lambda	15 min timeout; 128 MB–10 GB; 1 vCPU at 1,769 MB, 6 vCPUs at 10 GB; 6 MB sync / 1 MB async payload; 1,000 concurrency default; /tmp up to 10 GB; async retries 2; events kept 6 h
Firehose	Buffer: 1–128 MB and 0–900 s for S3; Lambda transform 6 MB response; delivers to Iceberg tables directly
IAM role session	15 minutes to 12 hours; role chaining max 1 hour

A.2 Ten traps' interviewers set

10 Common AWS Senior Data Engineer Interview Mistakes

1. Talking about S3 eventual consistency. S3 has been strongly consistent since December 2020.
2. Calling it "Kinesis Firehose" or implying replay. The current name is Amazon Data Firehose, and it is a delivery service, not a replayable event stream.
3. Putting all historical data in Redshift. Keep large historical data in S3, and use Athena, Redshift Spectrum, or Iceberg to query it when appropriate.
4. Forgetting kms:Decrypt in cross-account access. When encrypted S3 data uses a KMS key, the consuming role may need KMS key permissions, including kms:Decrypt, in addition to S3 permissions.
5. Forgetting iam:PassRole. When a user or service submits a Glue job or other AWS resource using an IAM role, the caller may need iam:PassRole to pass that role to the service.
6. Saying Glue Bookmarks automatically handle deletes or updates. Bookmarks primarily track processed data and state. They do not automatically detect and propagate source deletes or updates, so use CDC, MERGE, or another change-handling strategy.
7. Using interleaved sort keys as the default in Redshift. Prefer compound sort keys or AUTO based on the workload. Use interleaved only when its specific access pattern justifies it.
8. Designing one Lambda to process an entire S3 prefix. Lambda should handle small, bounded, event-driven work. For large prefixes or heavy processing, use SQS, Step Functions, Glue, or Spark with controlled parallelism.
9. Alarming primarily on CPU. For data pipelines, prioritize freshness, processing lag, failures, missing data, and data quality. CPU is usually a diagnostic metric rather than the business-level alarm.
10. Answering "streaming" for every latency requirement. First ask "What is the SLA?" Seconds may require streaming, minutes may be satisfied by micro-batch, and hourly or daily workloads are often better handled with batch.

11.A.3 Twenty-minute verbal drill

-
- Cover the answers and say each one out loud in under 60 seconds. If you stall, re-read the section's mental map - not the whole section.
 - Explain the three zones of an S3 data lake and what must never happen in the Raw zone.
 - Choose a storage class for hot ML features, nightly-read partitions, and a 7-year legal archive. Justify each choice in one sentence.
 - Explain DynamicFrame vs DataFrame and where each is used within a Glue job.
 - Explain why a Glue bookmark might not advance.
 - Read a Redshift EXPLAIN: what does DS_DIST_BOTH mean, and how would you fix it?
 - Explain when VACUUM still matters even with automatic vacuum.
 - Explain why partition projection exists and when Iceberg may be a better choice.
 - Explain the IAM policy evaluation order in one breath.
 - For a cross-account S3 read, which four policies/documents need the appropriate Allow?
 - Explain what the Kinesis partition key determines and what symptom indicates a hot shard.
 - Explain Amazon Data Firehose vs Kinesis Data Streams vs Managed Service for Apache Flink in one sentence each.
 - Give the Lambda concurrency formula and explain the two key concurrency settings.

- Explain when S3 → EventBridge is better than a direct S3 notification.
 - Name the four things you should primarily alarm on in a data pipeline.
 - Batch vs streaming: what question should you ask first?
 - Explain how you would handle late-arriving data in a daily Gold table without rebuilding the entire table.
-